UTEPSA · SAL-301 ALGORITMOS Y PROGRAMACIÓN

Análisis y diseño

Requerimientos, entradas-procesos-salidas, algoritmos y diagramas de flujo

Proyecto: As bajo la manga — Sistema web de reservas para barbería
Docente: Ing. Gloria M. Gonzales Fernandez
Estudiante: Eduard Gonzales
Octubre de 2026

Etapa 2 — Análisis y Diseño (06–09/10)

Evidencia exigida por el cronograma: **requerimientos + algoritmo + diagrama de flujo**. El cuerpo del documento (secciones 1 a 7) contiene los requerimientos, las reglas del negocio, la tabla de entradas-procesos-salidas y el algoritmo central con sus diagramas de flujo. Los anexos completan el resto de los algoritmos, el diseño preliminar (base de la Presentación 2 del 12/10), el cumplimiento de los requisitos del módulo, los riesgos y un glosario.

> Los datos marcados como **(referencial)** —horario, servicios y precios— se confirman con la barbería. El sistema los toma de la base de datos o de la configuración, así que cambiarlos no altera los algoritmos.

1. Actores

Actor	Descripción	Acceso
Cliente	Persona que reserva una cita desde la web. No necesita cuenta.	Público
Recepcionista	Persona que atiende el local o el teléfono: gestiona la agenda, registra citas por llamada o en el local y administra clientes.	Inicio de sesión, rol Recepcionista
Administrador	Dueño de la barbería. Todo lo de la recepcionista, más servicios, peluqueros, usuarios, reportes y registro de errores.	Inicio de sesión, rol Administrador
Servicio de correo (Brevo)	Sistema externo que entrega el correo de confirmación.	API (ver glosario)

2. Supuestos, restricciones y fuera de alcance

Código	Supuesto o restricción
S-01	La barbería atiende con **uno o más peluqueros** (actualización del 22/10; antes era un solo barbero). Cada peluquero tiene su propia agenda y el cliente elige con quién atenderse. El dueño usa el rol Administrador; el rol Recepcionista es para quien atiende el local o el teléfono. Los peluqueros no inician sesión: son datos que el Administrador registra.
S-02	El pago se hace en el local; el sistema solo registra el precio.
S-03	Si el personal cancela o reprograma una cita, avisa al cliente por teléfono. El correo de confirmación incluye el teléfono/WhatsApp de la barbería para que el cliente pida cambios.
S-04	Los cierres extraordinarios (feriados, enfermedad) no se configuran en el sistema: el personal cancela las citas afectadas.
S-05	La docente autorizó un sistema web con base de datos y correos. El "menú principal" pedido por el módulo es el **menú lateral del panel**, que cambia según el rol.
S-06	Los datos marcados como (referencial) se confirman con la barbería antes de la implementación.

Fuera de alcance: pagos en línea, aplicación móvil nativa, WhatsApp/SMS, recordatorios automáticos, sucursales, horarios distintos por peluquero, cuentas para clientes, bloqueo de días especiales (feriados, ausencias).

3. Reglas de negocio

Código	Regla
RN-01	Cada peluquero atiende una sola cita a la vez. Peluqueros distintos pueden atender a la misma hora (RN-21).

Código	Regla
RN-02	Horario de atención **(referencial)**: lunes a sábado; turno mañana 09:00–12:00 y turno tarde 14:00–20:00. Domingo cerrado.
RN-03	La agenda se divide en **bloques de 30 minutos**. Una cita solo empieza al inicio de un bloque.
RN-04	La duración de cada servicio es múltiplo de 30 minutos, entre 30 y 180.
RN-05	Una cita termina dentro del turno en que empieza: no cruza el descanso de 12:00 a 14:00 ni pasa de las 20:00.
RN-06	Dos citas **del mismo peluquero** no se superponen. Una cita nueva [horaInicio, horaFin) choca con una existente si `horaInicio < horaFinExistente` y `horaFin > horaInicioExistente`. Como los intervalos son semiabiertos (incluyen el inicio pero no el fin), las citas contiguas (10:00–10:30 y 10:30–11:00) sí se permiten.
RN-07	Ocupan horario las reservas **Confirmadas** o **Atendidas**. Las **Canceladas** y **No asistió** lo liberan.
RN-08	Se reserva desde hoy hasta **30 días** adelante. No se aceptan fechas pasadas ni domingos.
RN-09	Para el día de hoy: en la **web** solo se ofrecen horarios que empiecen al menos **30 minutos** después de la hora actual; en la **reserva manual** también se ofrece el bloque en curso (para el cliente que llega al local). La reprogramación usa la regla de la reserva manual.
RN-10	La hora de referencia es la de **Bolivia (UTC−4 fijo; Bolivia no usa horario de verano)**, aunque el servidor esté en Europa. El sistema nunca usa `DateTime.Now`: obtiene la hora con un único método, `Reloj.AhoraBolivia()` = `DateTime.UtcNow.AddHours(-4)`.
RN-11	El cliente se identifica por su **teléfono** (único, guardado sin +591). Si ya existe: en la web se reutiliza **sin cambiar sus datos** (solo se completa el correo si estaba vacío); en la reserva manual el personal actualiza los datos que ingresa. El correo de confirmación se envía a la dirección escrita en el formulario.
RN-12	El correo es **obligatorio en la web** (para la confirmación) y **opcional en la reserva manual**.
RN-13	Toda reserva se guarda en estado **Confirmada**: la confirmación es automática porque el horario ya se validó (es la acción "confirmar" del alcance de la propuesta). No existe un estado Pendiente. Estados: Confirmada → Atendida, Cancelada o No asistió (en código, `NoAsistio`). Solo una reserva Confirmada cambia de estado o se reprograma.
RN-14	**Cancelar** y **reprogramar** solo se permite si la cita aún no empezó. **Atendida** y **No asistió** solo si la cita ya empezó.
RN-15	El **precio se copia** a la reserva al crearla: si después cambia el precio del servicio, los reportes no se alteran.
RN-16	Los servicios **no se eliminan, se desactivan** (borrado lógico). Un servicio inactivo no aparece en la web.
RN-17	Un cliente con reservas **no se elimina**.
RN-18	Tras **3 intentos fallidos** de inicio de sesión, la cuenta se bloquea **5 minutos**. Siempre se muestra el mismo mensaje general, para no revelar qué usuarios existen.
RN-19	Siempre debe existir al menos **un Administrador activo**; un administrador no puede desactivarse a sí mismo.
RN-20	Si el correo de confirmación falla, la reserva **se mantiene**: el error va a `errores.txt`, la reserva queda con correo_enviado = falso y la agenda muestra el aviso "correo no enviado" para que el personal llame al cliente. Si la reserva no tiene correo, correo_enviado queda vacío y no hay aviso.
RN-21	Toda reserva tiene **un peluquero**. Los horarios disponibles se calculan con las reservas de ese peluquero: la cita de un peluquero no ocupa el horario de otro. Los peluqueros no se eliminan, se desactivan (como RN-16); debe quedar al menos uno activo. Si hay un solo peluquero activo, la web salta el paso de elegirlo.

Servicios (referencial):

Servicio	Duración	Precio
Corte de cabello	30 min	Bs 35
Perfilado de barba	30 min	Bs 25
Corte + barba	60 min	Bs 55
Diseño de cejas	30 min	Bs 15

Servicio	Duración	Precio
Tinte	90 min	Bs 120

4. Requerimientos funcionales

Código	Requerimiento	Actor	Prioridad
RF-01	Mostrar el catálogo de servicios activos con imagen, duración y precio.	Cliente	Alta
RF-02	Elegir una fecha en un calendario que deshabilite fechas pasadas, domingos y días a más de 30 días.	Cliente	Alta
RF-03	Calcular y mostrar los horarios disponibles del peluquero elegido en la fecha, según la duración del servicio; los no disponibles aparecen tachados.	Cliente	Alta
RF-04	Ingresar los datos de contacto: nombre, apellido, teléfono, correo y notas opcionales.	Cliente	Alta
RF-05	Mostrar el resumen de la cita (servicio, duración, fecha, hora, precio y cliente) y permitir editar o confirmar.	Cliente	Alta
RF-06	Registrar la reserva confirmada, verificando otra vez que el horario siga libre.	Sistema	Alta
RF-07	Enviar al cliente un correo de confirmación con el detalle de la cita y el teléfono de la barbería.	Sistema	Alta
RF-08	Iniciar y cerrar sesión con usuario y contraseña; mostrar el menú según el rol.	Recepcionista, Administrador	Alta
RF-09	Bloquear la cuenta 5 minutos tras 3 intentos fallidos (RN-18).	Sistema	Media
RF-10	Ver la agenda del día con todos los datos de cada cita, filtrando por fecha y estado.	Recepcionista, Administrador	Alta
RF-11	Ver la agenda de la semana.	Recepcionista, Administrador	Media
RF-12	Registrar una reserva manual (llamada o en el local) con el mismo cálculo de horarios; correo opcional.	Recepcionista, Administrador	Alta
RF-13	Cancelar una reserva confirmada que aún no empezó (libera el horario).	Recepcionista, Administrador	Alta
RF-14	Marcar como Atendida o No asistió una reserva que ya empezó.	Recepcionista, Administrador	Alta
RF-15	Reprogramar una reserva confirmada a otra fecha u hora disponible.	Recepcionista, Administrador	Media
RF-16	Registrar, buscar (por nombre o teléfono), modificar y eliminar clientes (solo sin reservas).	Recepcionista, Administrador	Alta
RF-17	Ver el historial de citas de un cliente.	Recepcionista, Administrador	Media
RF-18	Registrar, modificar, activar y desactivar servicios (nombre, descripción, duración, precio e imagen ya incluida en el sitio o URL; no se suben archivos).	Administrador	Media
RF-19	Crear, modificar y desactivar usuarios del personal, asignando el rol.	Administrador	Media
RF-20	Generar reportes por rango de fechas —citas por día y por estado, servicios más solicitados (ordenados), ingresos reales y estimados— y exportarlos a .txt.	Administrador	Media
RF-21	Exportar la agenda del día a un archivo de texto (`agenda_aaaammdd.txt`).	Recepcionista, Administrador	Alta

Código	Requerimiento	Actor	Prioridad
RF-22	Registrar los errores del sistema en `App_Data/errores.txt` con fecha, hora, origen y mensaje.	Sistema	Alta
RF-23	Ver en el panel el registro de errores (lee `errores.txt` y muestra las últimas 100 líneas, la más nueva primero).	Administrador	Alta
RF-24	Elegir el peluquero con quien atenderse (paso 2 de la reserva web); en la reserva manual, el personal lo elige. La agenda, la lista de reservas y la exportación se pueden filtrar por peluquero.	Cliente, Recepcionista, Administrador	Media (22/10)
RF-25	Registrar, modificar, activar y desactivar peluqueros (nombre, apellido, especialidad). Los reportes muestran citas e ingresos por peluquero.	Administrador	Media (22/10)

Plan por prioridad:

- **Alta = Versión funcional V1:** se implementa del 13 al 16/10 (con 17–18/10 de colchón) y se muestra en la Presentación 3 (19/10).
- **Media:** 19 y 20/10, después de la P3.
- **21–22/10:** solo pruebas y correcciones.
- La V1 arranca con **datos iniciales**: un script SQL con los 5 servicios referenciales y, si la tabla de usuarios está vacía, la aplicación crea un Administrador con contraseña temporal.

5. Requerimientos no funcionales

Código	Tipo	Requerimiento
RNF-01	Usabilidad	Interfaz en español, adaptable a celular (responsive); la reserva toma como máximo 5 pasos (servicio, peluquero, fecha y hora, datos, confirmar).
RNF-02	Seguridad	Las contraseñas se guardan con **hash** (huella que no se puede revertir) calculado con **PBKDF2** (`Rfc2898DeriveBytes.Pbkdf2`, SHA-256, 100 000 repeticiones) y una **sal** aleatoria de 16 bytes. La sal se guarda junto al hash en contrasena_hash (texto "sal:hash"). Para verificar, se recalcula el hash de la contraseña escrita con la misma sal y se comparan. Nunca se guardan en texto plano.
RNF-03	Seguridad	La sesión del personal se guarda en una **cookie** protegida (HttpOnly: ningún script de la página puede leerla; Secure: solo viaja por HTTPS) que vence a las 8 horas. Las páginas del panel exigen sesión y el rol correcto.
RNF-04	Seguridad	Los formularios llevan un código de verificación (token antiforgery) para que otra página no pueda enviar formularios falsos en nombre del usuario (CSRF). Las consultas a la base usan parámetros, nunca texto pegado, para evitar la inyección SQL. Las vistas muestran lo que escribe el usuario como texto, para que no pueda inyectar código en la página (XSS). Al usuario se le dan mensajes generales; el detalle técnico va solo a `errores.txt`. En Supabase se activa la seguridad por filas (RLS) sin reglas: nadie puede leer las tablas desde internet y el sistema entra con su propia cadena de conexión.
RNF-05	Seguridad	La cadena de conexión y la clave de Brevo no se escriben en el código ni se suben a GitHub: en la computadora de desarrollo se guardan en los secretos de usuario de .NET (user-secrets, un archivo fuera del proyecto) y en el hosting, en su panel de configuración.
RNF-06	Seguridad	Solo HTTPS: certificado gratuito del hosting y redirección automática de HTTP a HTTPS.
RNF-07	Integridad	No puede haber dos reservas superpuestas aunque dos personas confirmen al mismo tiempo. Se garantiza en dos niveles: (1) el sistema vuelve a verificar en C# antes de guardar; (2) la base de datos rechaza cualquier choque (Anexo B.3).
RNF-08	Confiabilidad	Los errores se capturan con `try/catch`, se muestra un mensaje claro y se registran en `errores.txt`. Si el propio registro falla, se ignora sin interrumpir al usuario.
RNF-09	Rendimiento	Las páginas responden en menos de 3 segundos con conexión normal, salvo la primera carga después de un tiempo sin uso. La llamada a Brevo espera como máximo 10 segundos.

Código	Tipo	Requerimiento
RNF-10	Disponibilidad	Publicado en internet (MonsterASP.NET) y disponible las 24 horas, dentro de lo que permiten los planes gratuitos.
RNF-11	Mantenibilidad	C# orientado a objetos, en dos proyectos (lógica y presentación), con nombres coherentes con el diagrama UML.
RNF-12	Formatos	Fechas dd/mm/aaaa, horas HH:mm (24 h), montos en bolivianos (Bs), hora de Bolivia (RN-10).

6. Entradas, procesos y salidas

N.º	Nombre	RF	Entradas	Proceso	Salidas
P-01	Consultar servicios	RF-01	—	Leer los servicios activos.	Tarjetas de servicios con imagen, duración y precio.
P-02	Calcular horarios disponibles	RF-02, RF-03, RF-12, RF-24	Fecha, servicio, peluquero, reserva a excluir, origen	Validar la fecha; recorrer los bloques de cada turno y descartar los que ya pasaron o chocan con una reserva de ese peluquero (A1 y A1.1).	Lista de horarios disponibles (el resto, tachado).
P-03	Registrar reserva	RF-04, RF-05, RF-06, RF-12, RF-24	Servicio, peluquero, fecha, hora, nombre, apellido, teléfono, correo, notas, origen	Validar; volver a verificar el horario; en una transacción, buscar o crear el cliente y guardar la reserva Confirmada; manejar el choque de horario (A2).	Pantalla "¡Cita confirmada!" y reserva guardada.
P-04	Enviar correo de confirmación	RF-07	Reserva, correo del cliente	Armar el mensaje (servicio, fecha, hora, precio y teléfono de la barbería); enviarlo por la API de Brevo (máximo 10 s); guardar la marca correo_enviado; registrar el error si falla.	Correo recibido, o aviso "correo no enviado".
P-05	Iniciar y cerrar sesión	RF-08, RF-09	Usuario, contraseña	Validar campos; buscar el usuario; verificar bloqueo y hash; contar intentos (A3). Cerrar sesión elimina la cookie y vuelve al inicio.	Panel según el rol, o mensaje general.
P-06	Ver y exportar agenda	RF-10, RF-11, RF-21	Fecha o semana, estado (filtro)	Consultar las reservas del rango ordenadas por hora. Para exportar, armar el texto en memoria y enviarlo como descarga.	Tabla con cliente, teléfono, correo, servicio, hora, precio, estado, notas, origen y aviso "correo no enviado" (solo si correo_enviado = falso); archivo `agenda_aaaammdd.txt`.
P-07	Cambiar estado	RF-13, RF-14	Reserva, acción	Verificar estado y hora (RN-13, RN-14); pedir confirmación; guardar (A4).	Estado actualizado; horario libre si se cancela.

N.º	Nombre	RF	Entradas	Proceso	Salidas
P-08	Reprogramar	RF-15	Reserva, nueva fecha, nueva hora	Calcular horarios excluyendo la propia reserva; verificar la hora; actualizar fecha y horas (A4).	Reserva actualizada o mensaje de error.
P-09	Gestionar clientes	RF-16, RF-17	Datos del cliente o texto de búsqueda	Validar; registrar, buscar (recorrido lineal), modificar o eliminar (solo sin reservas); listar su historial.	Lista de clientes, historial de citas, mensajes.
P-10	Gestionar servicios	RF-18	Nombre, descripción, duración, precio, imagen, activo	Validar; guardar; activar o desactivar.	Catálogo actualizado.
P-11	Gestionar usuarios	RF-19	Usuario, nombre, apellido, contraseña, rol, activo	Validar; calcular el hash; guardar; mantener al menos un Administrador activo.	Usuario creado o actualizado.
P-12	Generar reporte y exportar	RF-20	Fecha de inicio, fecha de fin	Recorrer las reservas del rango contando por día, estado y servicio; sumar ingresos; ordenar los servicios por burbuja; armar el .txt en memoria (A5 y A5.1).	Reporte en pantalla y archivo `reporte_aaaammdd_aaaammdd.txt` descargado.
P-13	Registrar error	RF-22	Origen, mensaje de la excepción	Agregar una línea "fecha hora · origen · mensaje" a `App_Data/errores.txt`, de a una escritura por vez.	Línea agregada en `errores.txt`.
P-14	Ver registro de errores	RF-23	—	Leer `errores.txt` línea por línea; tomar las últimas 100; si no existe, informarlo.	Lista de errores en pantalla, o "Sin errores registrados".
P-15	Gestionar peluqueros	RF-25	Nombre, apellido, especialidad, activo	Validar; registrar o modificar; activar o desactivar (al menos uno activo, RN-21).	Lista de peluqueros; la web muestra solo los activos.

Validaciones de los datos de entrada:

Dato	Regla
Nombre, apellido	Obligatorios; de 2 a 50 caracteres; solo letras y espacios (se aceptan tildes y ñ).
Teléfono	Obligatorio; 8 dígitos que empiezan con 6 o 7 (celular de Bolivia); se acepta el prefijo +591, que se quita antes de guardar; no puede repetirse al registrar o modificar un cliente (RN-11).
Correo	Obligatorio en la web y opcional en la reserva manual; si se ingresa: formato usuario@dominio.ext, máximo 100 caracteres.
Notas	Opcional; máximo 200 caracteres.
Fecha	Según RN-02 y RN-08.
Hora	Debe estar en la lista que devuelve A1 (RN-03 a RN-09).
Servicio (registro y edición)	Nombre obligatorio y único; precio mayor que 0; duración múltiplo de 30, entre 30 y 180.

Dato	Regla
Peluquero	Obligatorio en toda reserva; debe existir y estar activo. Nombre y apellido con la regla de arriba; especialidad opcional, máximo 60 caracteres.
Usuario (registro y edición)	Nombre de usuario único de 4 a 20 caracteres; contraseña de al menos 8 caracteres con letras y números; rol Administrador o Recepcionista.

7. Algoritmos

> **Notación.** `←` es asignación. Se usan las estructuras **secuencial**, **condicional** (Si … si no; Según) y **repetitivas** (Mientras; Para cada; Repetir … hasta que). En la web, un "Repetir hasta que los datos sean válidos" significa que el formulario se vuelve a mostrar con el mensaje: cada intento es una nueva petición al controlador. Los ciclos internos (turnos, bloques, reservas, recorrido del reporte, burbuja) son `while`, `for` y `foreach` en C#. Los algoritmos de la lógica **devuelven** resultados; los mensajes los muestra la página (el controlador).

> **Símbolos de los diagramas de flujo.** Óvalo: inicio o fin. Paralelogramo: entrada o salida. Rectángulo: proceso. Rombo: decisión. Rectángulo con barras laterales: subalgoritmo. Círculo: conector (un círculo con el mismo número o letra une una salida con su continuación, dentro del mismo diagrama o en otra parte del mismo algoritmo). Texto rojo: mensaje de error.

Esta sección presenta el algoritmo central del sistema (A1, con su prueba de escritorio y su subalgoritmo A1.1) y el registro de una reserva (A2). Los algoritmos A3 a A5.1 están en el **Anexo A**.

A1. Calcular horarios disponibles

Implementa: RF-02, RF-03, RF-12, RF-24 · **Reglas:** RN-01 a RN-10, RN-21 **Método:** `GestorReservas.CalcularHorariosDisponibles(fecha, servicio, idPeluquero, idExcluir, origen)` **Parámetros:** F (fecha), S (servicio), P (id del peluquero), idExcluir (entero: id de una reserva que no se cuenta, usado al reprogramar; 0 = ninguna), origen (Web o Manual). **Devuelve:** disponibles (lista de horas de inicio). **Variables:** ahora y limite (fecha y hora), hoy (fecha), fechaValida (lógico), d (entero, minutos), R (lista de Reserva), h (hora), choca (lógico).

1. Inicio.
2. Recibir F, S, P, idExcluir y origen.
3. ahora ← Reloj.AhoraBolivia(); hoy ← fecha de ahora; disponibles ← lista vacía.
4. fechaValida ← (F ≥ hoy) y (F ≤ hoy + 30 días) y (F no es domingo).
5. Si fechaValida es falso: devolver disponibles (vacía) y terminar.
6. d ← duración de S en minutos.
7. Si origen = Web: limite ← ahora + 30 minutos. Si no: limite ← inicio del bloque en curso (ahora redondeado hacia abajo a un múltiplo de 30 minutos; por ejemplo, 10:10 → 10:00).
8. R ← reservas del día F **del peluquero P** con estado Confirmada o Atendida y id ≠ idExcluir.
9. Para cada turno T en [mañana 09:00–12:00, tarde 14:00–20:00]:
    1. h ← inicio de T.
    2. Mientras h + d ≤ fin de T:
        1. Si F + h ≥ limite (el bloque todavía no pasó):
            1. choca ← HayChoque(h, d, R); si choca es falso: agregar h a disponibles.
        2. h ← h + 30 minutos.
10. Devolver disponibles.
11. Fin.

La página muestra todos los bloques del horario y tacha los que no están en disponibles; si la lista está vacía, muestra «No hay horarios disponibles para esta fecha».

Complejidad: O(b × r), donde b = bloques del día (18 como máximo) y r = reservas del día (filtrarlas por peluquero es un recorrido más, O(r)).

Prueba de escritorio de A1:

Datos: F = lunes 12/10/2026 (consultado un día antes, así que el bloque nunca "ya pasó"), S = Corte + barba (d = 60 min), P = Luis, origen = Web, R = {10:00–11:00, 15:00–16:30} (reservas de Luis; si otro peluquero tuviera una cita a las 09:00, no entraría en R).

Turno	**h**	**h + d**	**¿Entra en el turno?**	**¿HayChoque?**	**Resultado**
Mañana	09:00	10:00	Sí	No (10:00 no es mayor que 10:00)	Disponible
Mañana	09:30	10:30	Sí	Sí, con 10:00–11:00	—
Mañana	10:00	11:00	Sí	Sí, con 10:00–11:00	—
Mañana	10:30	11:30	Sí	Sí, con 10:00–11:00	—
Mañana	11:00	12:00	Sí	No (11:00 no es menor que 11:00)	Disponible
Mañana	11:30	12:30	No: termina el Mientras	—	—
Tarde	14:00	15:00	Sí	No (15:00 no es mayor que 15:00)	Disponible
Tarde	14:30	15:30	Sí	Sí, con 15:00–16:30	—
Tarde	15:00	16:00	Sí	Sí, con 15:00–16:30	—
Tarde	15:30	16:30	Sí	Sí, con 15:00–16:30	—
Tarde	16:00	17:00	Sí	Sí, con 15:00–16:30	—
Tarde	16:30	17:30	Sí	No (16:30 no es menor que 16:30)	Disponible
Tarde	17:00 a 19:00	18:00 a 20:00	Sí	No	Disponibles (5 horarios)
Tarde	19:30	20:30	No: termina el Mientras	—	—
Resultado					**9 horarios:** 09:00, 11:00, 14:00, 16:30, 17:00, 17:30, 18:00, 18:30 y 19:00

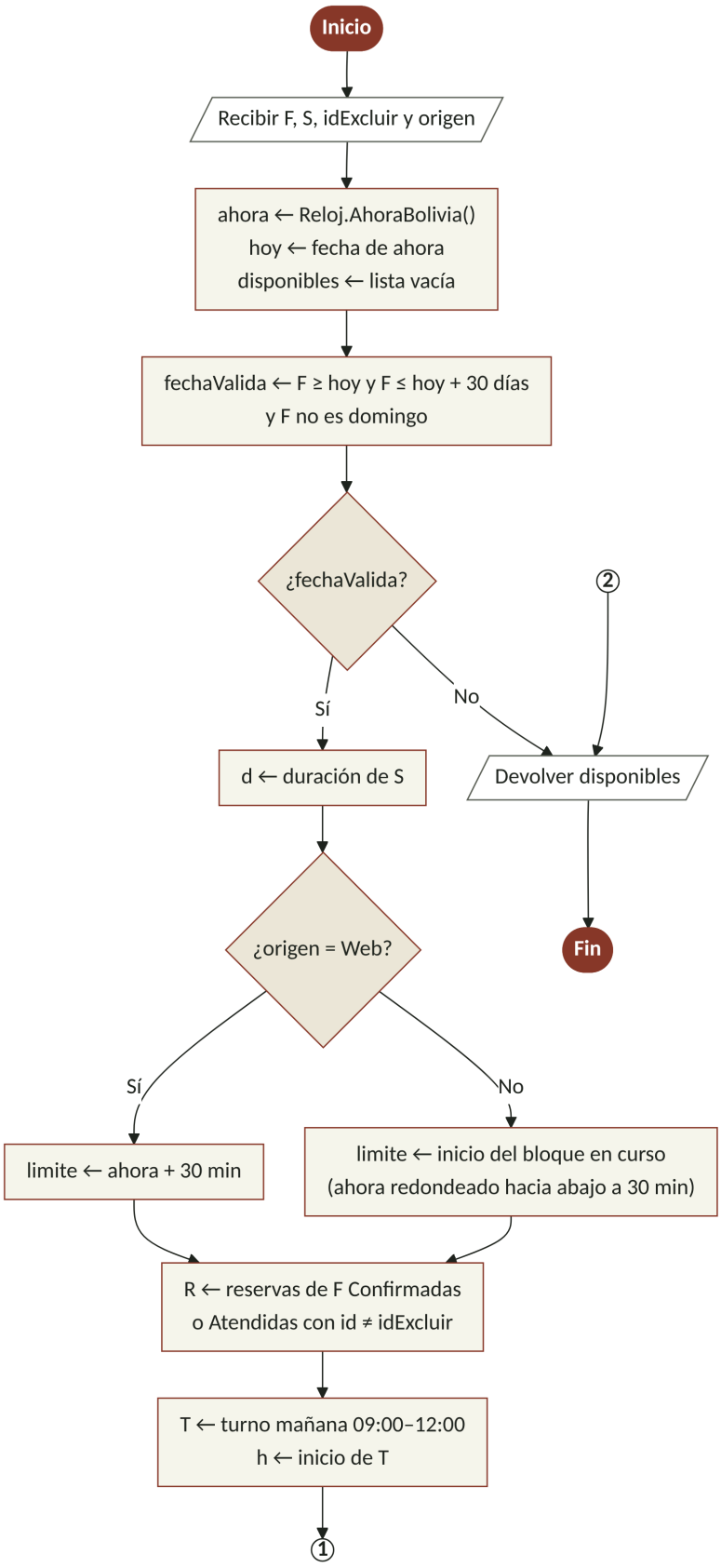


Figura 1. A1 — Calcular horarios disponibles (parte 1: validación y preparación)

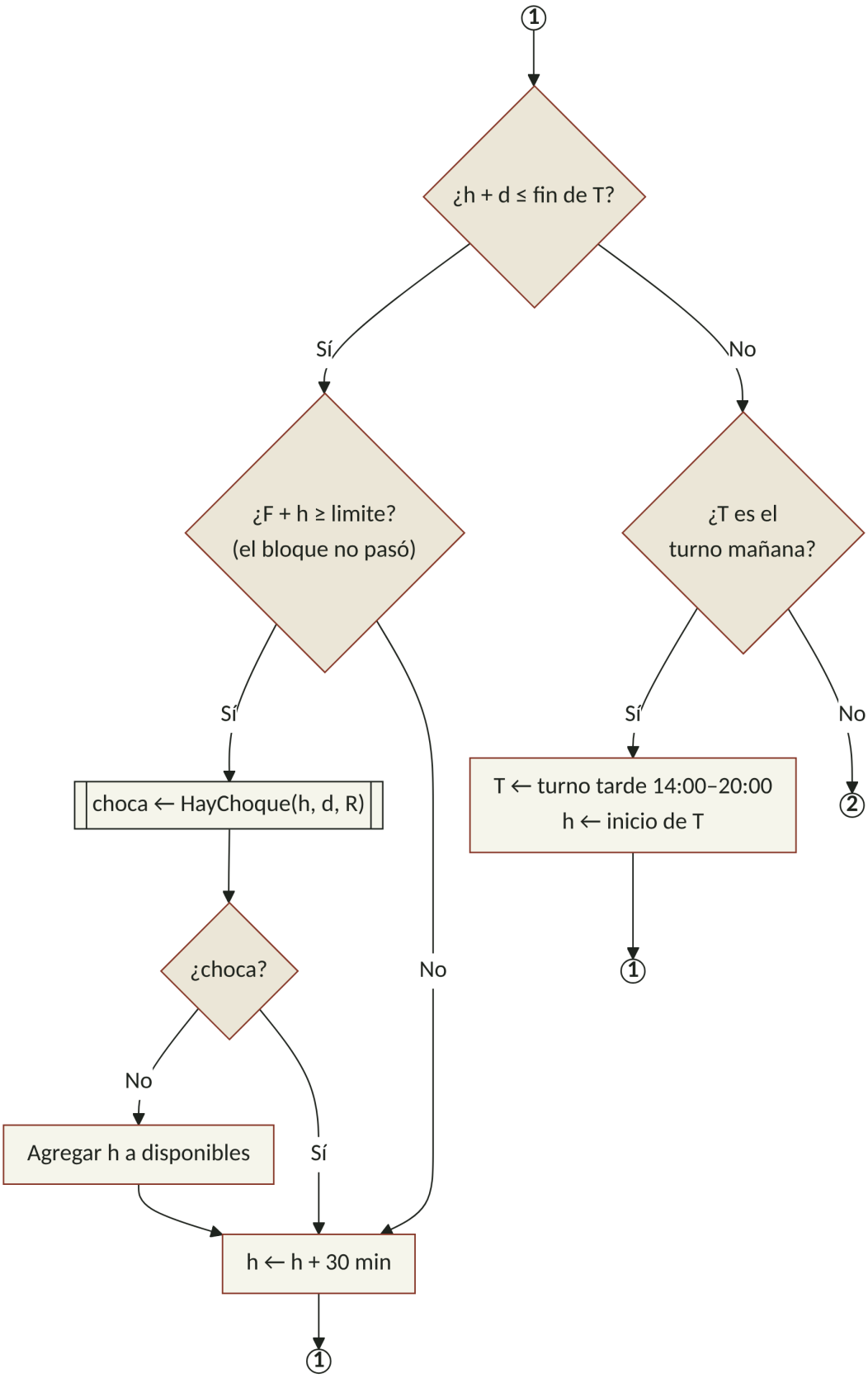


Figura 2. A1 — Calcular horarios disponibles (parte 2: recorrido de los bloques)

A1.1. Subalgoritmo HayChoque

Implementa: RN-06 · **Método:** `GestorReservas.HayChoque(h, d, R)` **Parámetros:** h (hora de inicio propuesta), d (duración en minutos), R (lista de reservas del día). **Devuelve:** choca (lógico). **Variables:** fin (hora), i (entero).

1. Inicio.

2. Recibir h, d y R.
3. fin ← h + d; choca ← falso; i ← 0.
4. Mientras i < cantidad de R y choca = falso:
    1. Si h < R[i].horaFin y fin > R[i].horaInicio: choca ← verdadero.
    2. i ← i + 1.
5. Devolver choca.
6. Fin.

Complejidad: O(r): en el peor caso recorre todas las reservas del día.

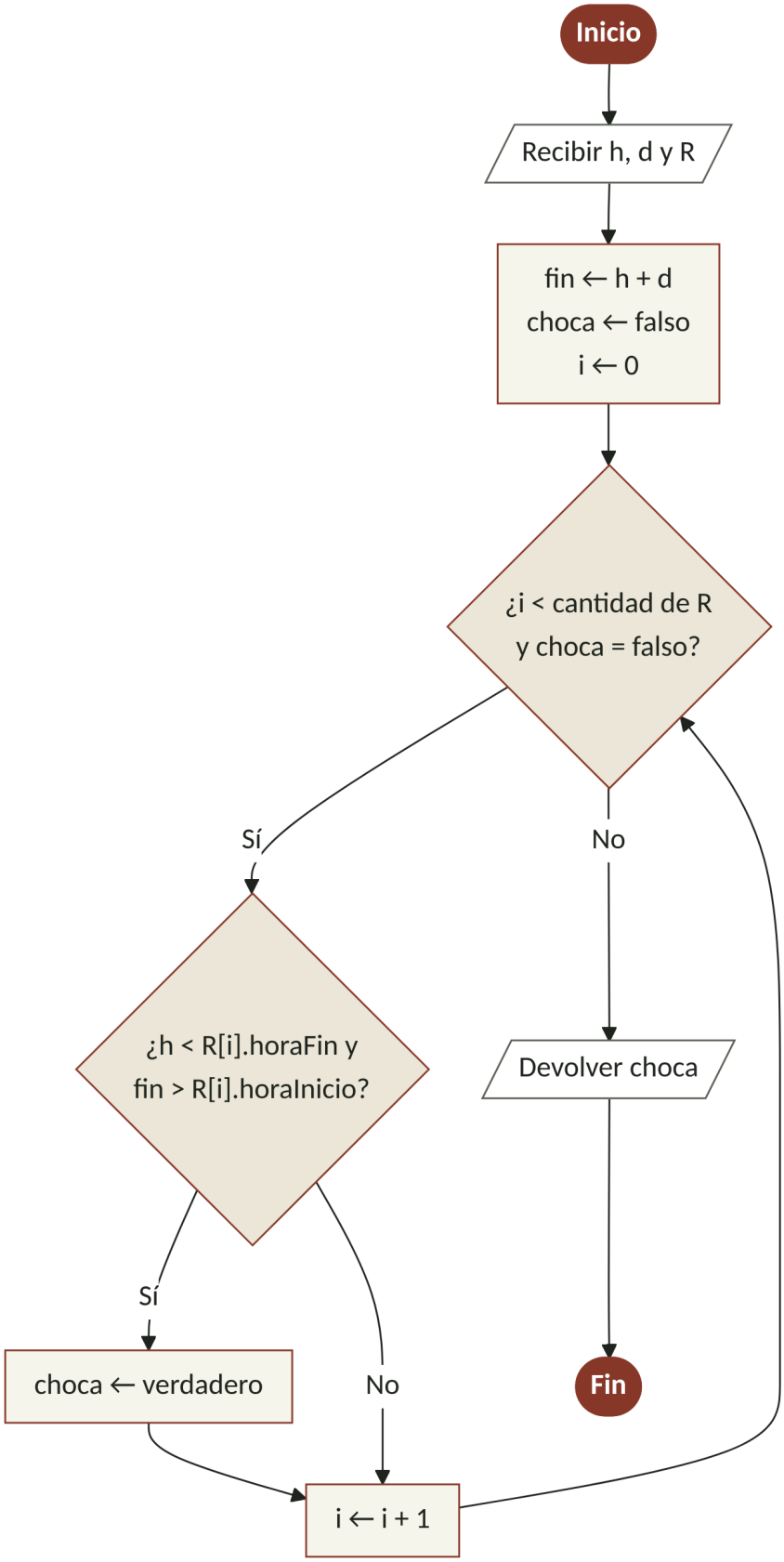


Figura 3. A1.1 — Subalgoritmo HayChoque

A2. Registrar reserva (web o manual)

Implementa: RF-04 a RF-07, RF-12 · **Reglas:** RN-11 a RN-13, RN-15, RN-20 **Método:** `GestorReservas.RegistrarReserva(...)` **Entradas:** origen (Web o Manual) y, del usuario: servicio S, fecha F, hora H, nombre, apellido, teléfono, correo, notas. En la reserva manual, el usuario de la sesión. **Salida:** reserva

guardada y pantalla de confirmación. **Variables:** disponibles (lista de horas), errores (lista de textos), cliente (Cliente), res (Reserva).

1. Inicio.
2. Mostrar los servicios activos (P-01) y leer el servicio S. Mostrar los peluqueros activos y leer el peluquero P (si hay uno solo, se elige solo).
3. Repetir: leer la fecha F; disponibles ← A1(F, S, P, 0, origen); si disponibles está vacía: mostrar «No hay horarios disponibles para esta fecha». Hasta que disponibles no esté vacía.
4. Mostrar todos los bloques del horario (los que no están en disponibles, tachados) y leer la hora H.
5. Repetir: leer nombre, apellido, teléfono, correo y notas; errores ← Validar(datos, origen); si errores no está vacía: mostrar errores. Hasta que errores esté vacía.
6. Mostrar el resumen. Si se elige «Editar»: volver al paso 2 conservando lo ya ingresado. Si se elige «Confirmar»: seguir.
7. disponibles ← A1(F, S, P, 0, origen). Si H no está en disponibles: mostrar «Ese horario ya no está disponible, elige otro» y volver al paso 3 (la fecha queda preseleccionada).
8. En una transacción de la base de datos (todo se guarda junto o nada):
    1. Buscar el cliente por teléfono (sin +591).
    2. Si no existe: crear el cliente. Si existe y origen = Web: solo completar el correo si estaba vacío. Si existe y origen = Manual: actualizar los datos que vengan con valor.
    3. Crear res: peluquero ← P; horaInicio ← H; horaFin ← H + duración de S; precio ← precio de S; estado ← Confirmada; origen ← el recibido; creadoEn ← Reloj.AhoraBolivia(); creadoPor ← usuario de la sesión (solo Manual); correoEnviado ← falso si se ingresó correo (vacío si no).
    4. Guardar res y confirmar la transacción.
9. Si el guardado falla:
    1. Deshacer la transacción.
    2. Si la base de datos lo rechazó por choque de horario (código 23P01; se lanza HorarioNoDisponibleException): mostrar «Ese horario ya no está disponible, elige otro» y volver al paso 3.
    3. Si es otro error: registrarlo en `errores.txt`, mostrar «No se pudo registrar la reserva, intenta nuevamente» y terminar.
10. Si se ingresó correo:
    1. Intentar enviar el correo de confirmación por Brevo (máximo 10 segundos).
    2. Si se envió: correoEnviado ← verdadero. Si falló: registrar el error en `errores.txt`.
    3. Actualizar correo_enviado de res en la base de datos.
11. Mostrar «¡Cita confirmada!» con el detalle. Si correoEnviado es verdadero, agregar «Se envió el correo de confirmación (si no llega, revisar Spam)». Si es falso, agregar «No se pudo enviar el correo de confirmación». (Mensajes válidos para la web y para la reserva manual.)
12. Fin.

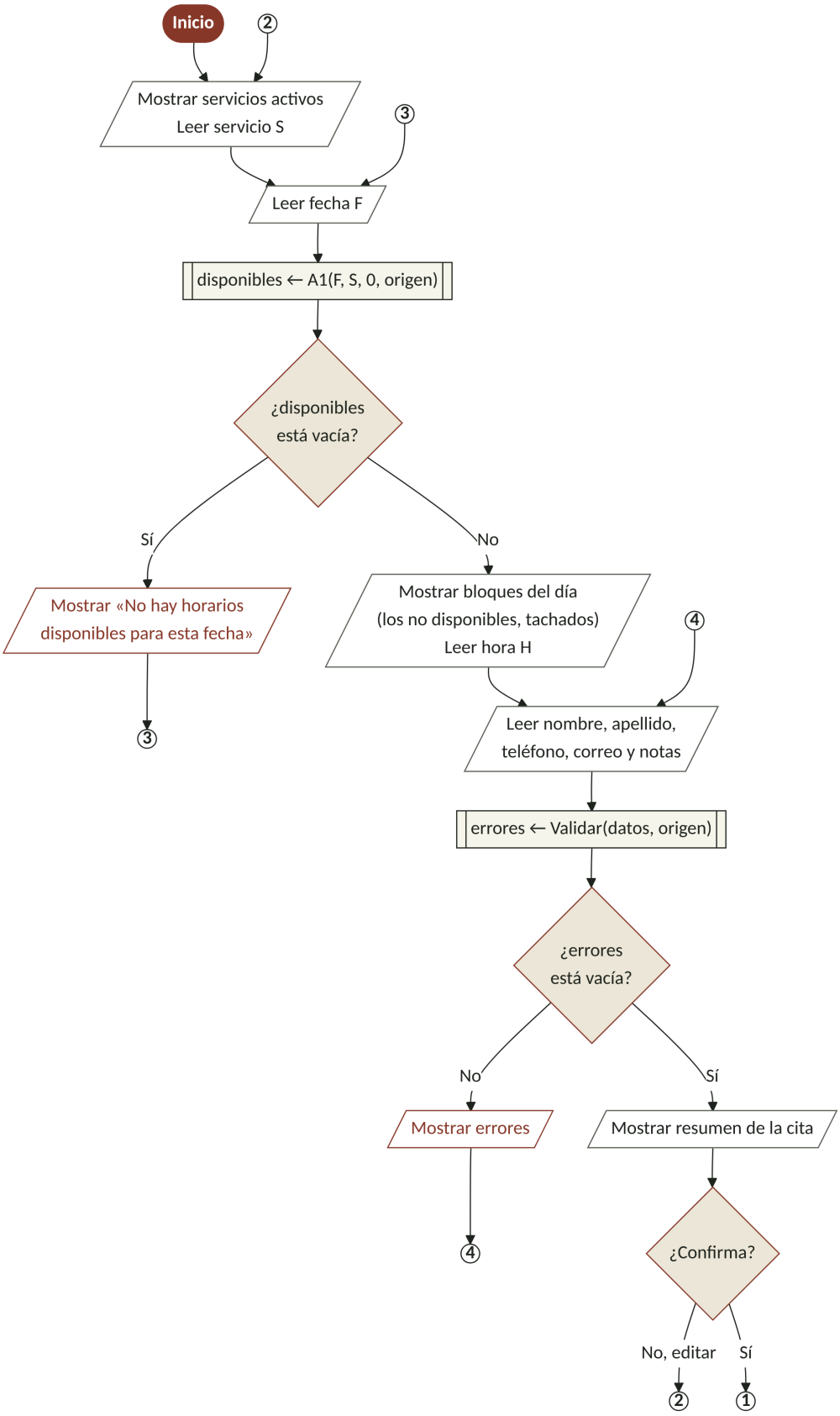


Figura 4. A2 — Registrar reserva (parte 1: servicio, fecha, hora y datos)

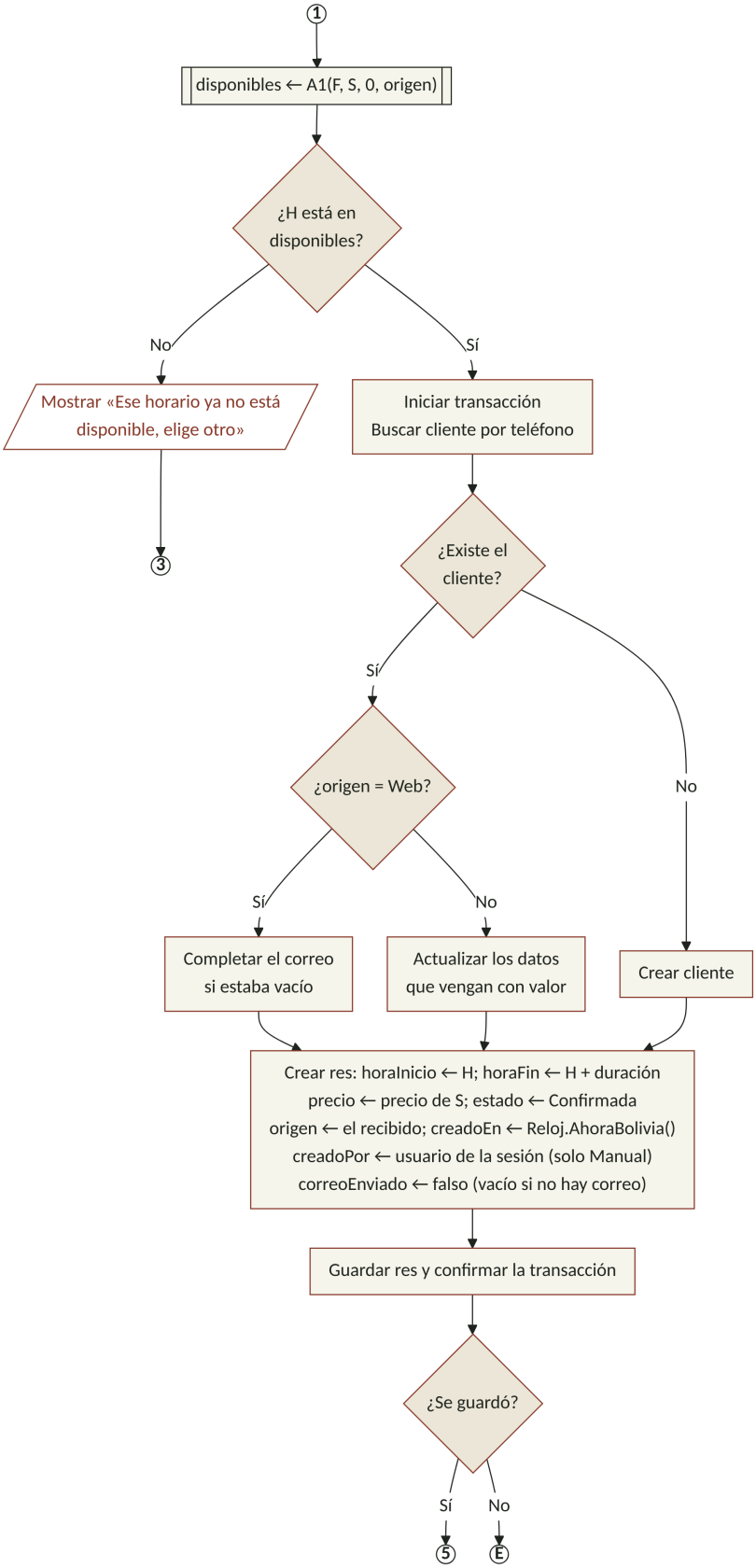


Figura 5. A2 — Registrar reserva (parte 2: verificación y guardado)

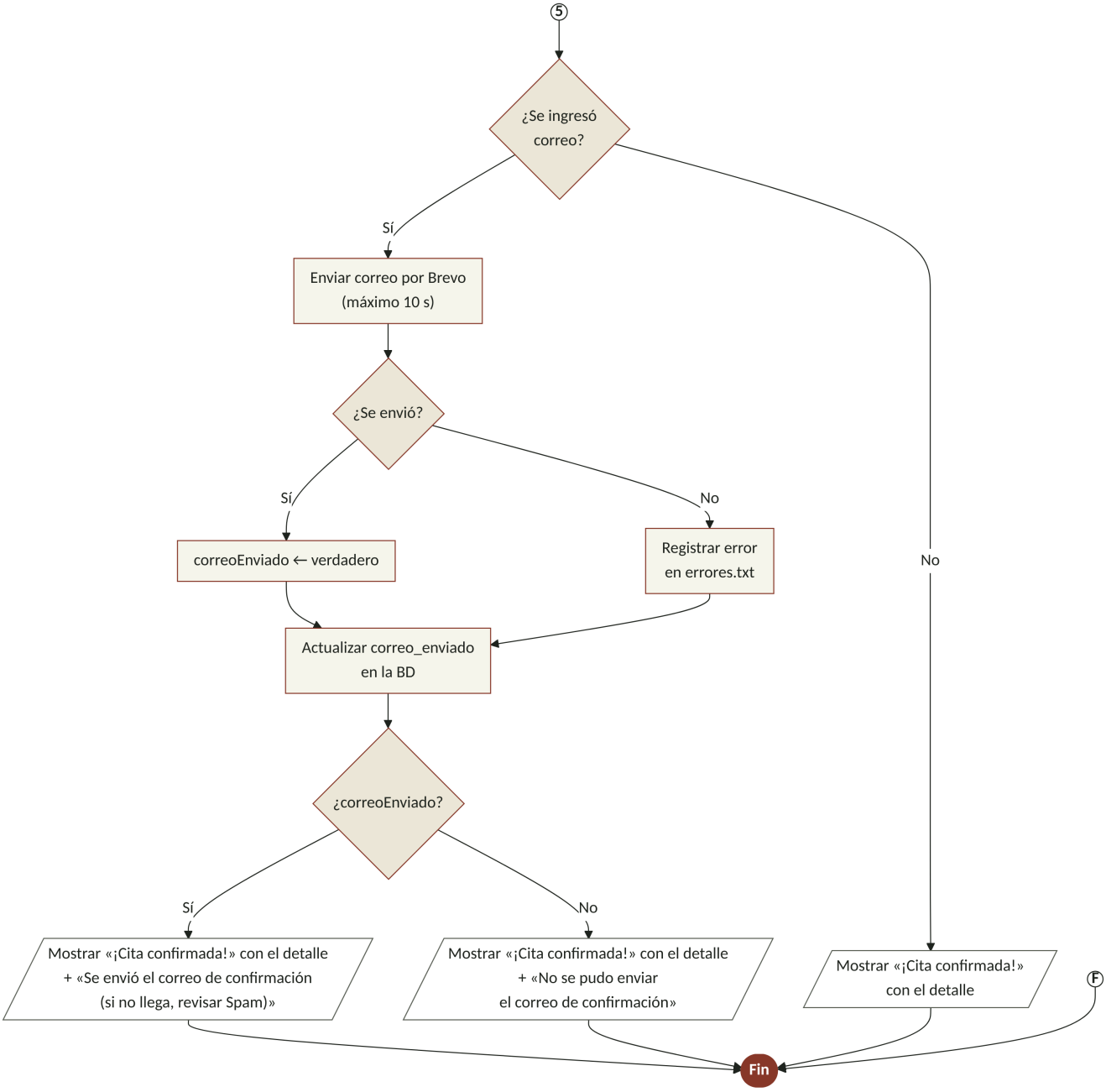


Figura 6. A2 — Registrar reserva (parte 3: correo de confirmación)

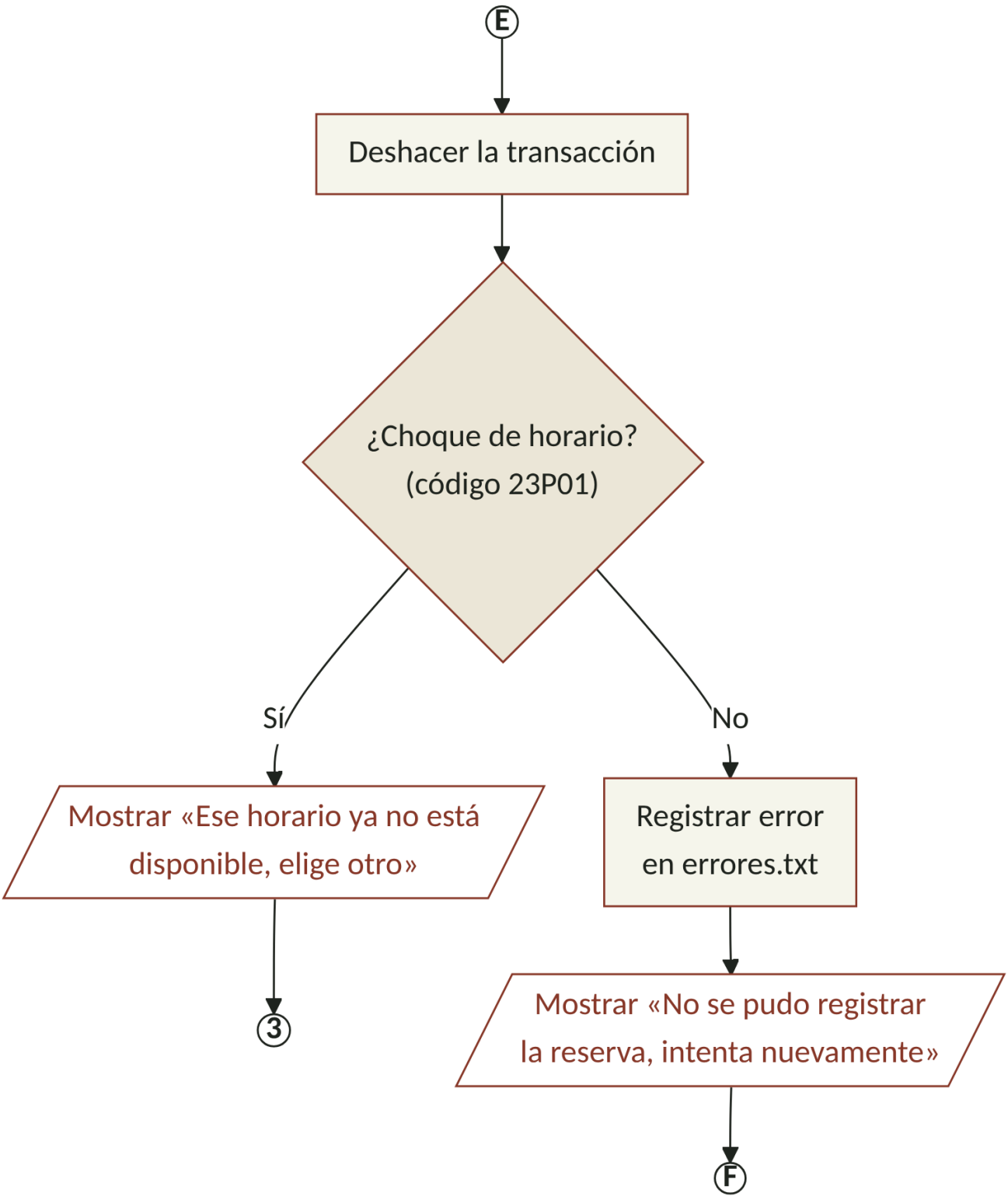


Figura 7. A2 — Registrar reserva (parte 4: error al guardar)

Anexo A. Algoritmos complementarios

A3. Iniciar sesión

Implementa: RF-08, RF-09 · **Reglas:** RN-18 **Método:** `GestorUsuarios.IniciarSesion(usuario, contrasena)` **Entradas:** usuario U y contraseña C. **Salida:** panel según el rol. **Variables:** usuario (Usuario), ahora (fecha y hora), accesoConcedido (lógico). MENSAJE = «Usuario o contraseña incorrectos. Tras 3 intentos fallidos la cuenta se bloquea 5 minutos.»

1. Inicio.
2. accesoConcedido ← falso.
3. Repetir hasta que accesoConcedido sea verdadero:
    1. Leer U y C.
    2. Si U o C están vacíos: mostrar «Completa usuario y contraseña».
    3. Si no:
        1. ahora ← Reloj.AhoraBolivia(); buscar el usuario U.
        2. Si no existe, está inactivo o bloqueadoHasta > ahora: mostrar MENSAJE.
        3. Si no, si VerificarHash(C, contrasenaHash) es verdadero (contrasenaHash incluye la sal): intentosFallidos ← 0; bloqueadoHasta ← vacío; guardar el usuario; crear la sesión con su rol; accesoConcedido ← verdadero.
        4. Si no: intentosFallidos ← intentosFallidos + 1; si intentosFallidos ≥ 3: bloqueadoHasta ← ahora + 5 min e intentosFallidos ← 0; guardar el usuario; mostrar MENSAJE.
4. Si el rol es Administrador: mostrar el panel con el menú completo; si no: mostrar el panel de recepción.
5. Fin.

El contador de intentos y el bloqueo se guardan en la tabla usuarios (no en una variable), porque en la web cada intento es una petición nueva.

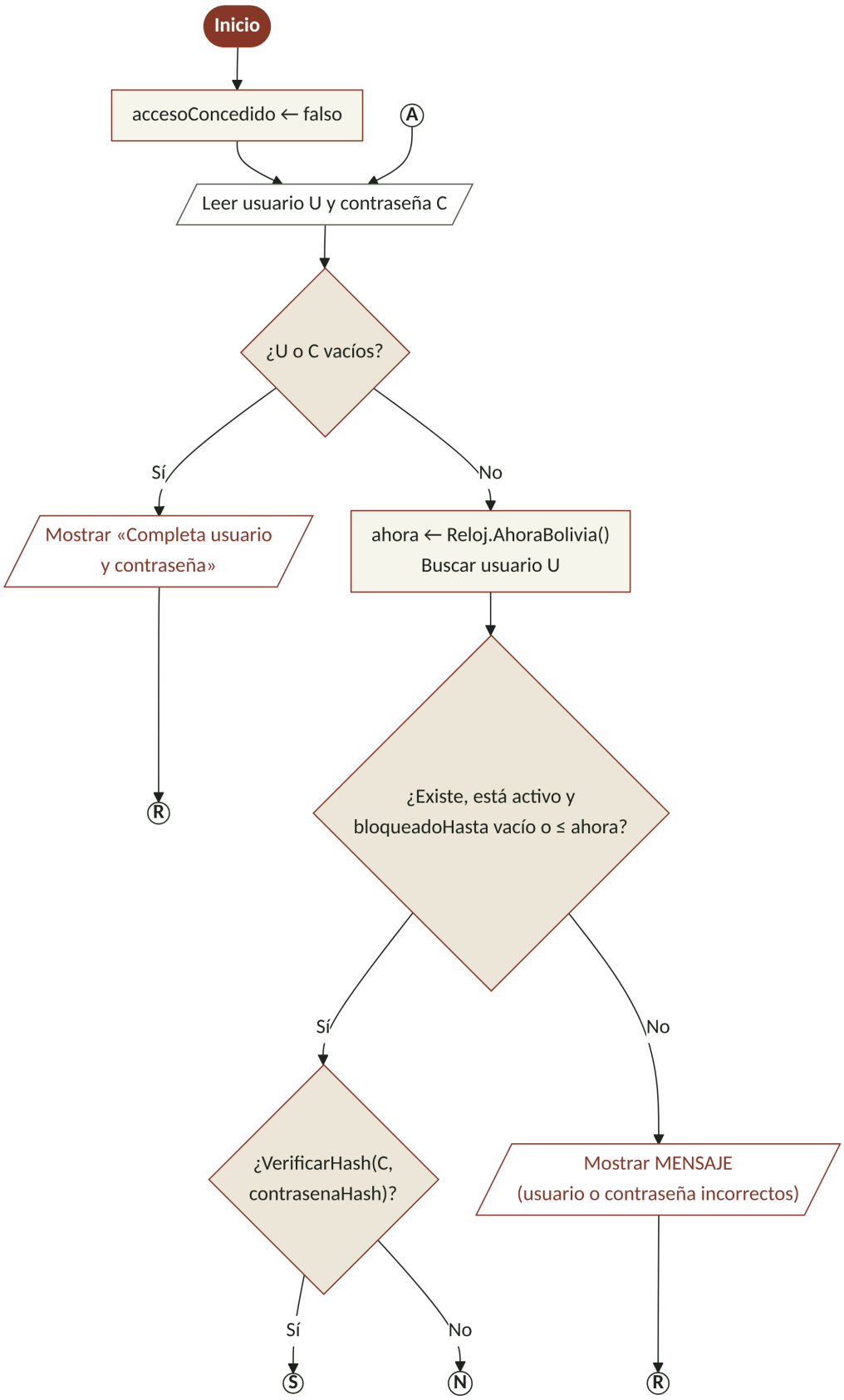


Figura 8. A3 — Iniciar sesión (parte 1: lectura y verificación)

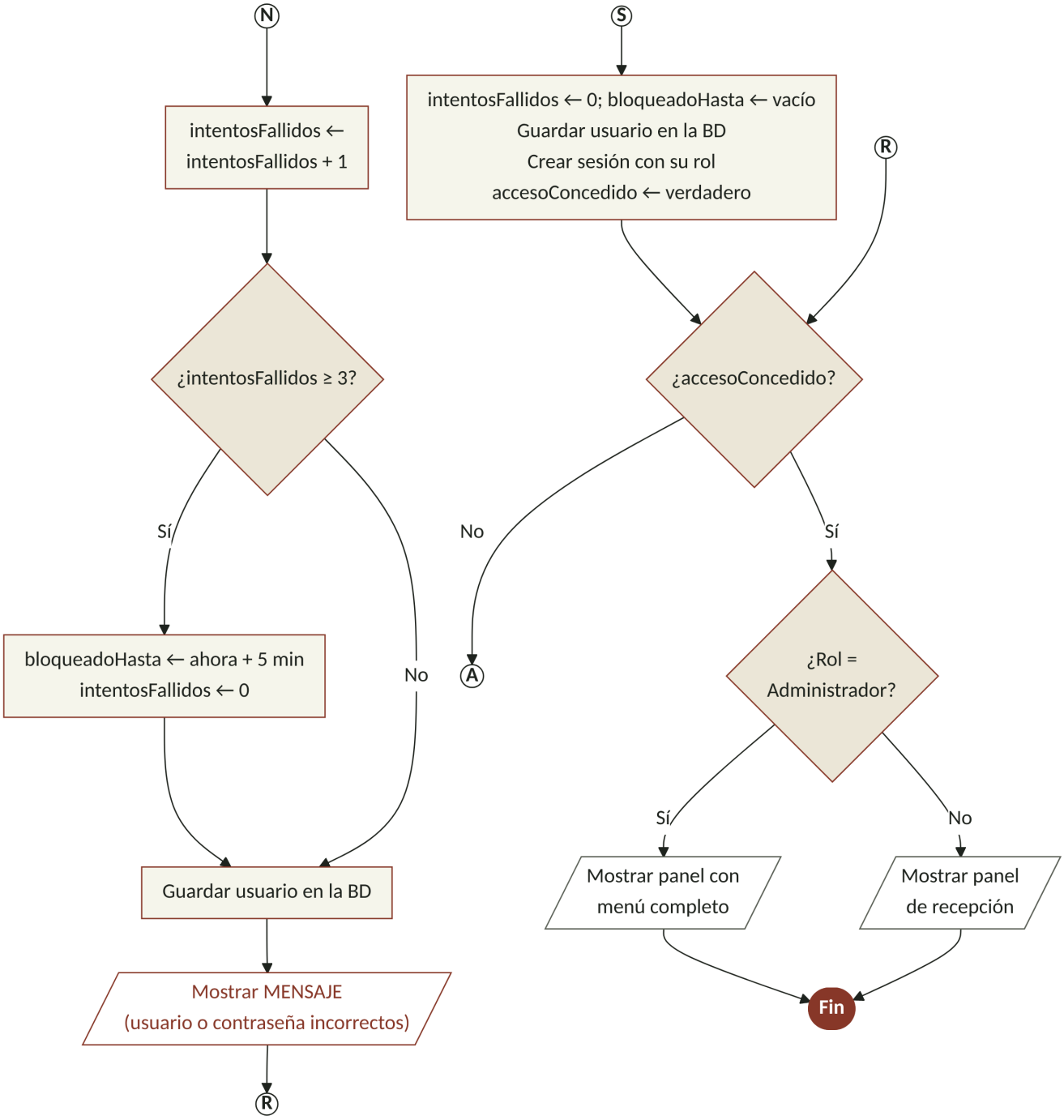


Figura 9. A3 — Iniciar sesión (parte 2: intento fallido o acceso)

A4. Gestionar una reserva (cambiar estado o reprogramar)

Implementa: RF-13, RF-14, RF-15 · **Reglas:** RN-13, RN-14 **Métodos:** `GestorReservas.CambiarEstado(...)` y `GestorReservas.Reprogramar(...)` **Entradas:** res (reserva elegida) y la acción A: Cancelar, Atendida, NoAsistio o Reprogramar. **Variables:** ahora e inicio (fecha y hora), disponibles (lista de horas), F2 (fecha), H2 (hora).

1. Inicio.
2. Leer res y A. ahora ← Reloj.AhoraBolivia(); inicio ← fecha de res + horaInicio de res.
3. Si el estado de res no es Confirmada: mostrar «Solo se pueden modificar reservas confirmadas» y terminar.
4. Según A:
    1. **Cancelar:** si inicio ≤ ahora: mostrar «La cita ya empezó: márcala como Atendida o No asistió» y terminar. Pedir confirmación; si no acepta, terminar. Estado de res ← Cancelada.

2. **Atendida o NoAsistio:** si inicio > ahora: mostrar «La cita aún no empezó» y terminar. Pedir confirmación; si no acepta, terminar. Estado de res ← A.
3. **Reprogramar:**
    1. Si inicio ≤ ahora: mostrar «La cita ya empezó» y terminar.
    2. Repetir: leer F2; disponibles ← A1(F2, servicio de res, peluquero de res, id de res, Manual); si disponibles está vacía: mostrar «No hay horarios para esa fecha». Hasta que disponibles no esté vacía.
    3. Leer H2. Volver a calcular disponibles con A1; si H2 no está en disponibles: mostrar «Ese horario ya no está disponible» y volver al paso 4.3.2.
    4. Fecha de res ← F2; horaInicio ← H2; horaFin ← H2 + duración del servicio.
5. Intentar guardar res.
6. Si falla por choque de horario (HorarioNoDisponibleException): mostrar «Ese horario ya no está disponible» y volver al paso 4.3.2. Si falla por otro error: registrarlo en `errores.txt`, mostrar «No se pudo guardar» y terminar.
7. Si se guardó: mostrar «Cambios guardados» y actualizar la agenda.
8. Fin.

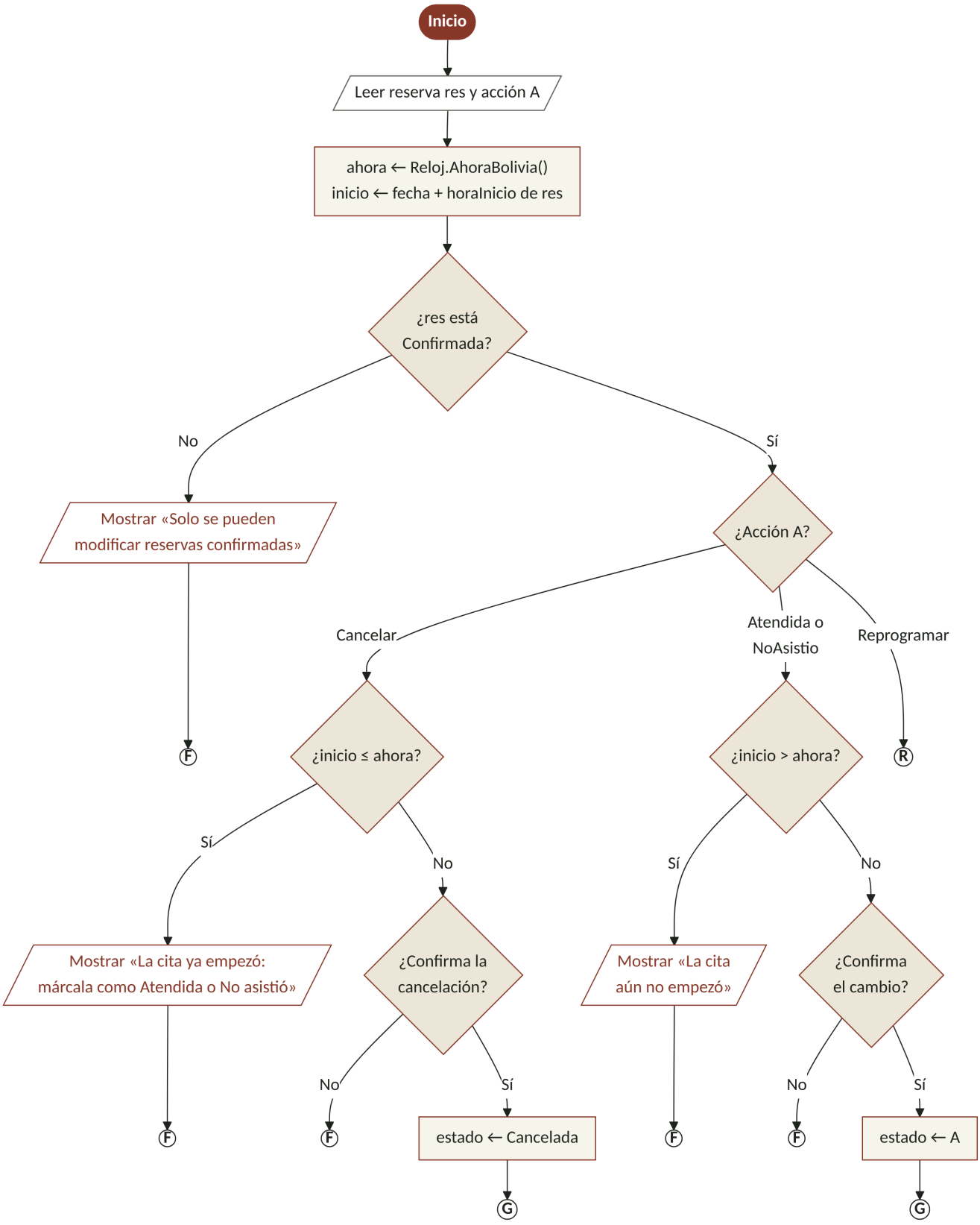


Figura 10. A4 — Gestionar una reserva (parte 1: cancelar, atendida o no asistió)

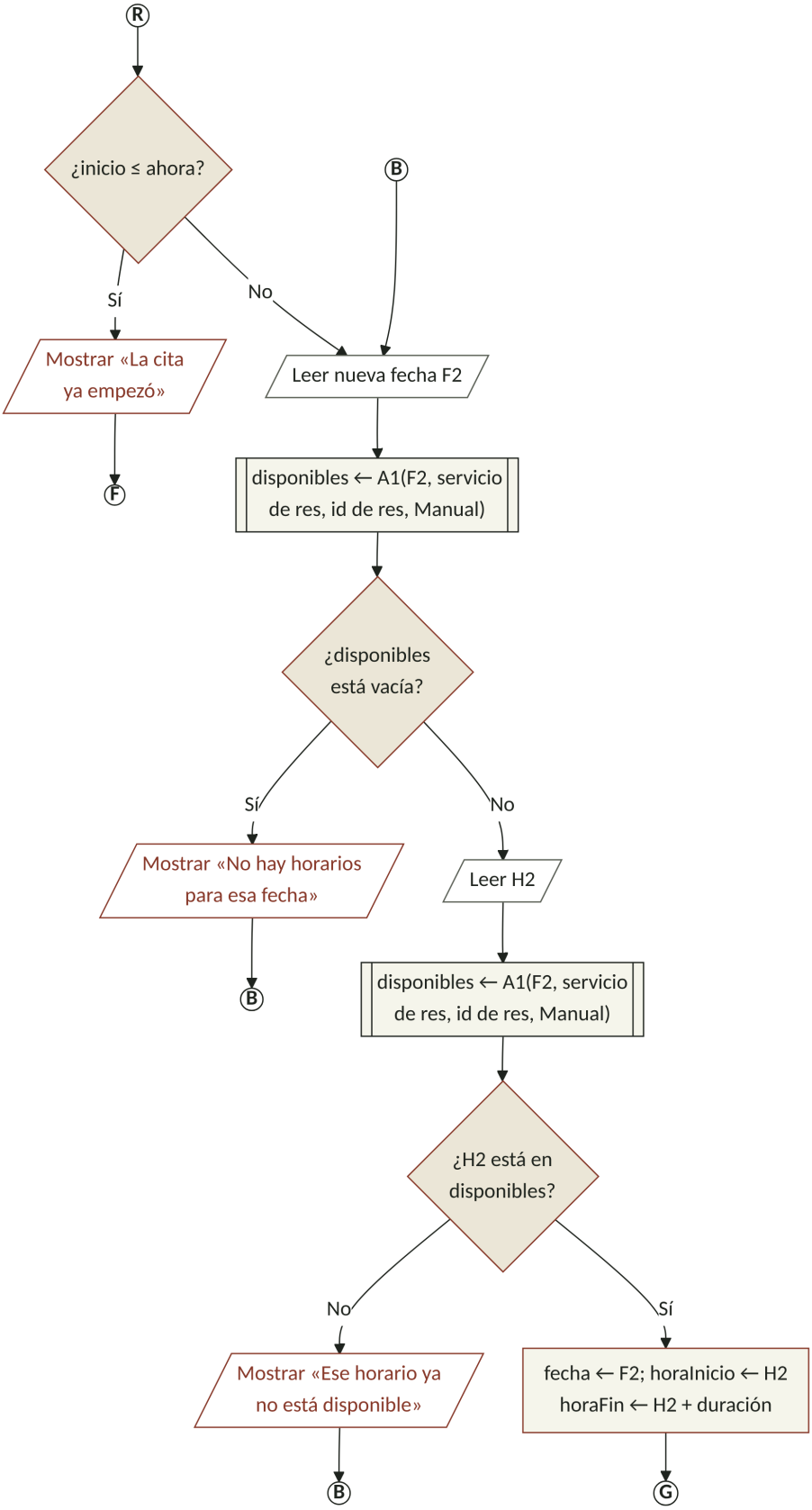


Figura 11. A4 — Gestionar una reserva (parte 2: reprogramar)

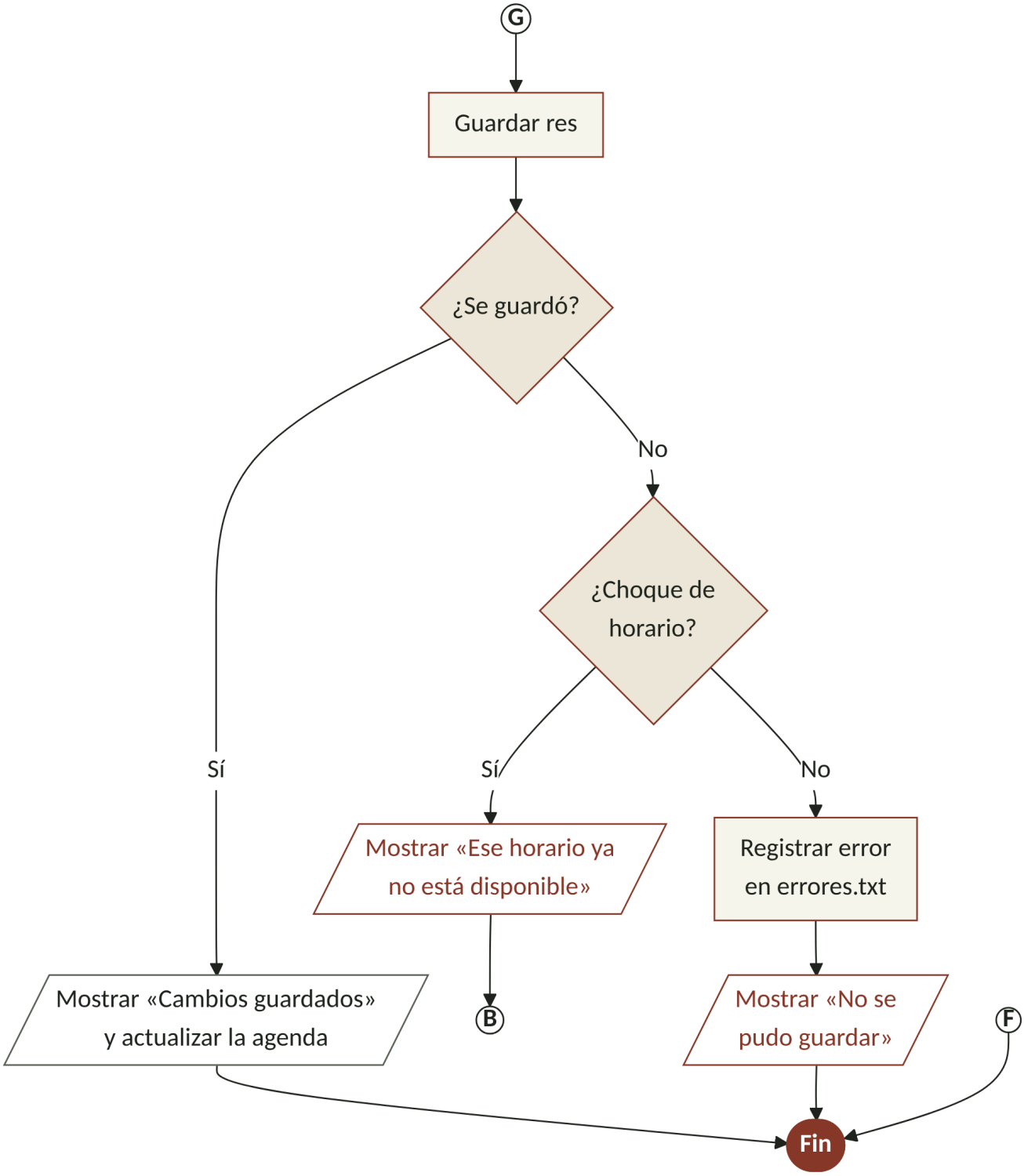


Figura 12. A4 — Gestionar una reserva (parte 3: guardar)

A5. Generar reporte y exportar a .txt

Implementa: RF-20 · **Reglas:** RN-07, RN-15 **Métodos:** `GeneradorReportes.Generar(fi, ff)` y `GeneradorReportes.ExportarTxt(...)` **Entradas:** fecha de inicio Fi y fecha de fin Ff. **Salida:** reporte en pantalla y archivo .txt. **Variables:** hoy (fecha), L (lista de Reserva), total (entero), contEstado (diccionario estado → entero), contDia (diccionario fecha → entero), contServicio (diccionario servicio → entero; se pasa a lista para ordenar), ingresoReal e ingresoEstimado (decimal, Bs), sinMarcar (entero).

1. Inicio.
2. Repetir: leer Fi y Ff; si Fi > Ff: mostrar «Rango de fechas no válido». Hasta que Fi ≤ Ff.

3. hoy ← fecha actual de Bolivia; L ← reservas con Fi ≤ fecha ≤ Ff (ambas fechas incluidas).
4. Si L está vacía: mostrar «No hay reservas en el rango» y terminar.
5. total ← 0; contEstado de los 4 estados ← 0; contDia y contServicio vacíos; ingresoReal ← 0; ingresoEstimado ← 0; sinMarcar ← 0.
6. Para cada reserva r en L:
    1. total ← total + 1; contEstado[estado de r] ← contEstado[estado de r] + 1; contDia[fecha de r] ← contDia[fecha de r] + 1.
    2. Si r está Atendida: ingresoReal ← ingresoReal + precio de r.
    3. Si r está Confirmada: si su fecha ≥ hoy: ingresoEstimado ← ingresoEstimado + precio de r; si no: sinMarcar ← sinMarcar + 1.
    4. Si r no está Cancelada: contServicio[servicio de r] ← contServicio[servicio de r] + 1.
7. Ordenar contServicio de mayor a menor cantidad con el método de burbuja (A5.1).
8. Mostrar el reporte: total, citas por estado, citas por día, servicios ordenados, ingresos y citas sin marcar.
9. Si se elige «Exportar»: armar el texto del reporte en memoria y enviarlo como descarga `reporte_aaaammdd_aaaammdd.txt` (con try/catch; si falla, registrar el error en `errores.txt` y mostrar «No se pudo exportar»). No se guarda nada en el servidor.
10. Fin.

Complejidad: $O(n + k^2)$, donde n = reservas del rango (se recorren una vez y cada suma en un diccionario es directa) y k = cantidad de servicios, por el ordenamiento de burbuja. Como la barbería tiene pocos servicios (5 de referencia), el costo de la burbuja es despreciable frente a n.

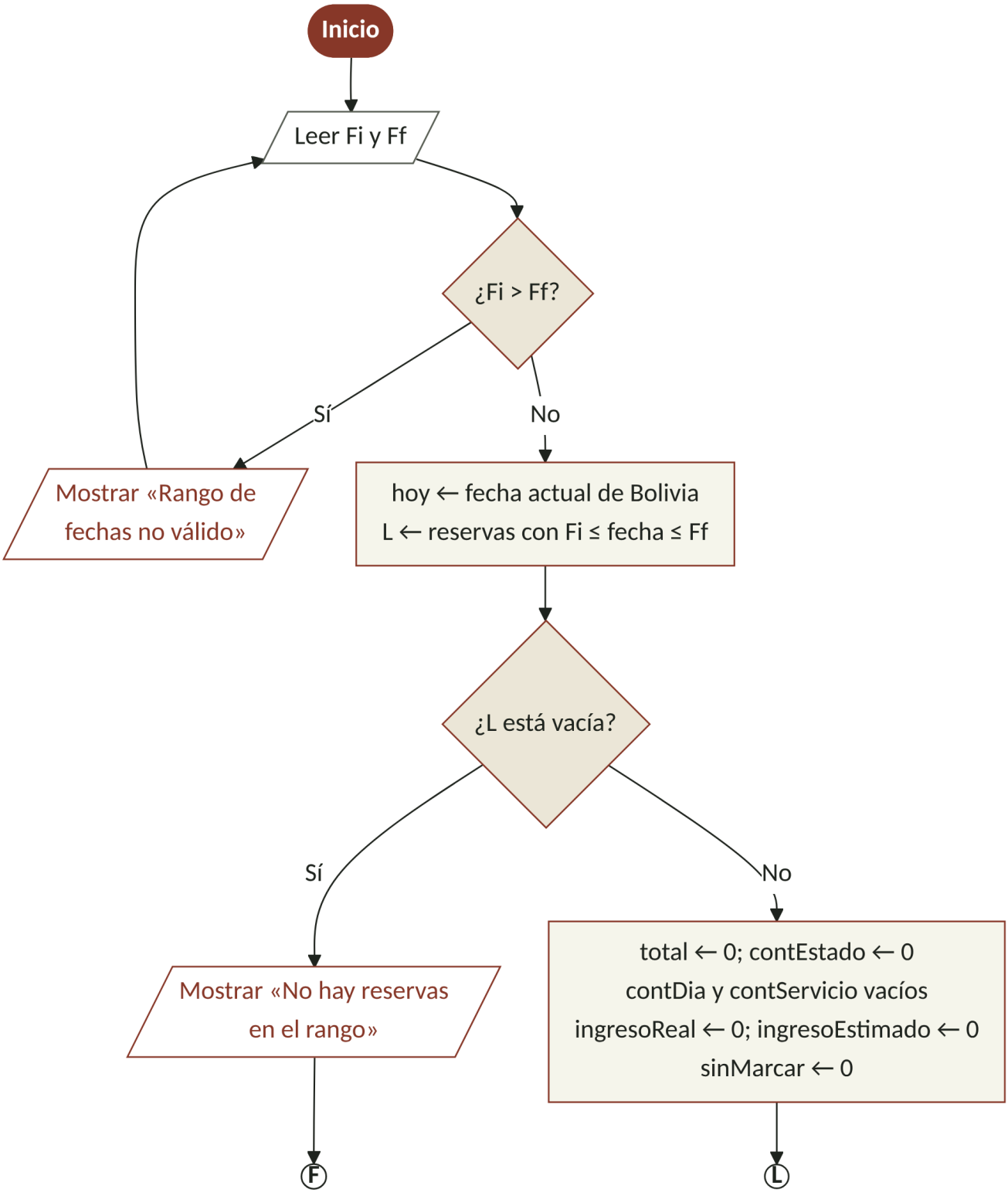


Figura 13. A5 — Generar reporte (parte 1: rango e inicialización)

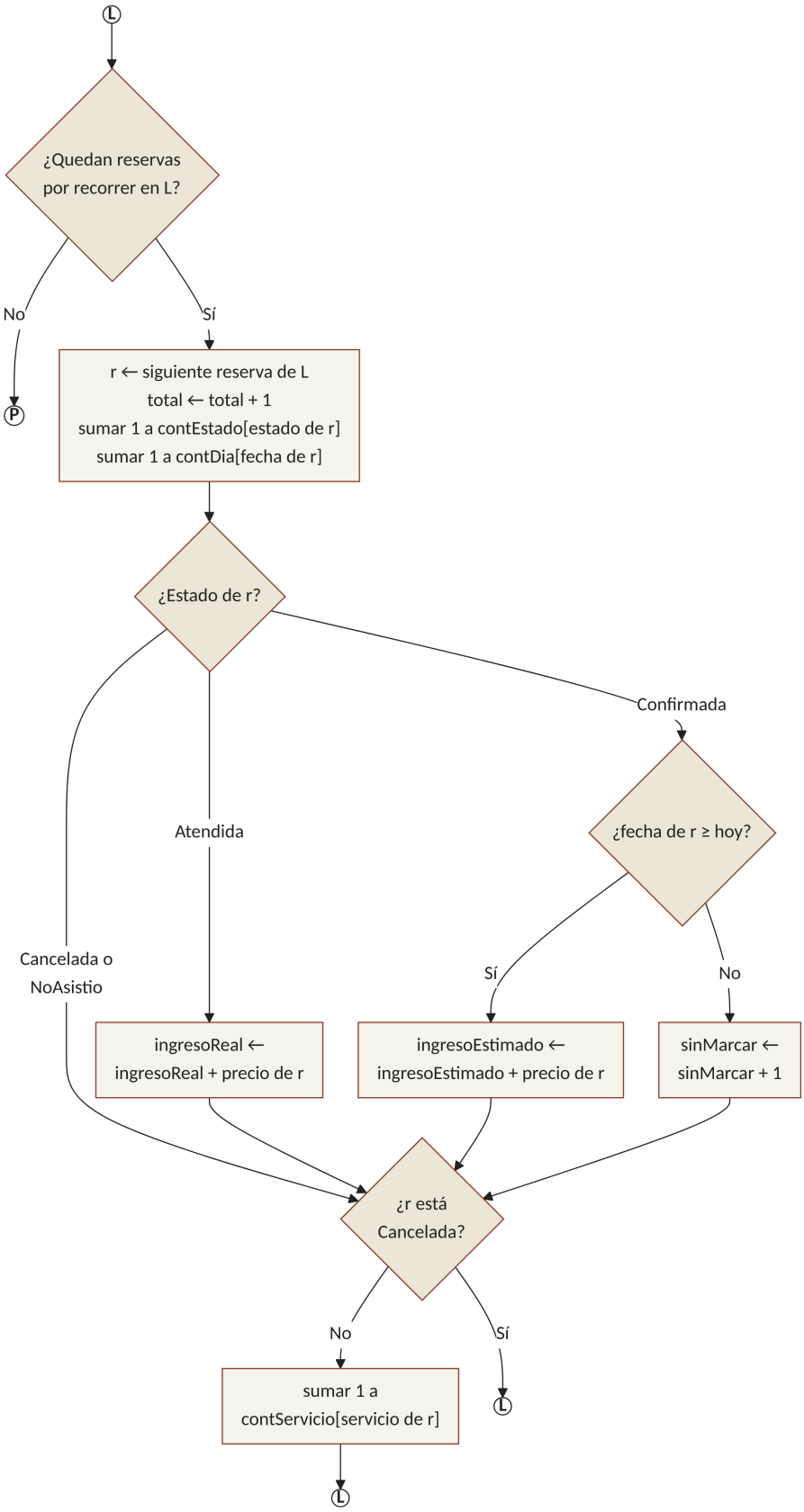


Figura 14. A5 — Generar reporte (parte 2: recorrido de las reservas)

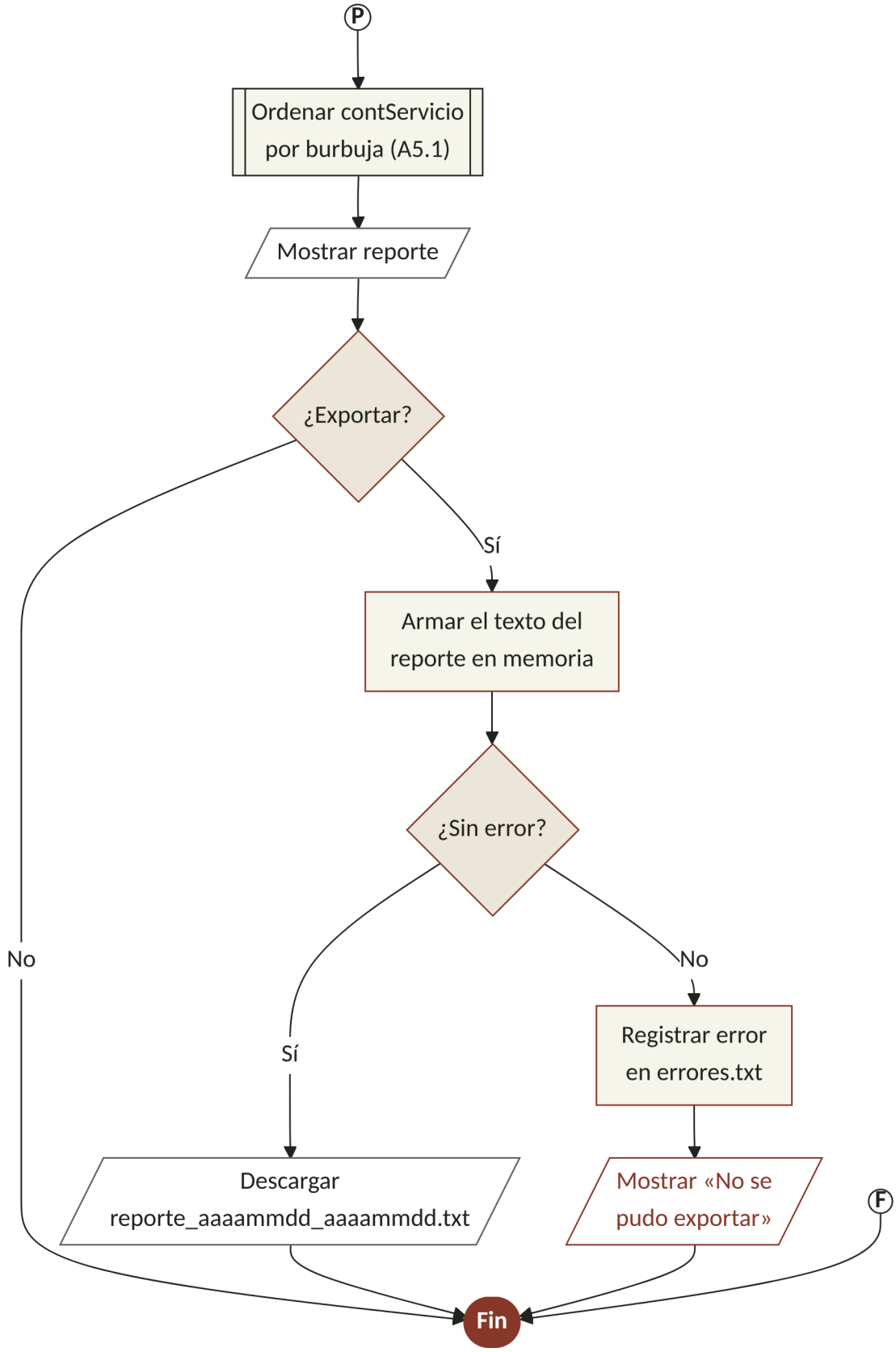


Figura 15. A5 — Generar reporte (parte 3: ordenar, mostrar y exportar)

A5.1. Ordenamiento de burbuja

Método: `GeneradorReportes.OrdenarBurbuja(lista)` **Parámetros:** lista de pares (servicio, cantidad). **Variables:** k, i, j (enteros), aux (par).

1. Inicio.

2. Recibir la lista de pares (servicio, cantidad).
3. k ← cantidad de elementos de la lista.
4. Para i desde 0 hasta k − 2:
    1. Para j desde 0 hasta k − 2 − i:
        1. Si lista[j].cantidad < lista[j + 1].cantidad: aux ← lista[j]; lista[j] ← lista[j + 1]; lista[j + 1] ← aux.
5. Fin.

Complejidad: $O(k^2)$ comparaciones.

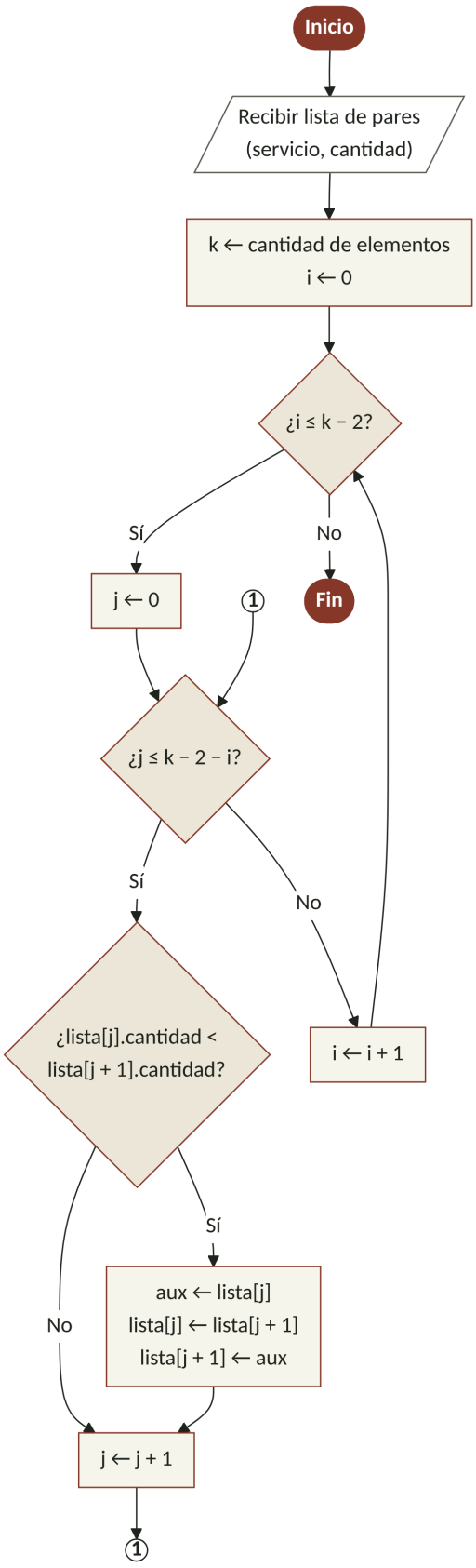


Figura 16. A5.1 — Ordenamiento de burbuja

Anexo B. Diseño preliminar de la solución

B.1 Arquitectura

- Dos proyectos, como en clase: `logica` (biblioteca de clases) y `presentaciones/WebBarberia` (ASP.NET Core MVC).
- El **navegador** llama a los **controladores**; estos usan los **gestores** de `logica`, que aplican las reglas y algoritmos y acceden a **Supabase**.
- `logica` también envía correos por **Brevo** y escribe `App_Data/errores.txt`. Los reportes y la agenda en .txt se arman en memoria y se descargan en el navegador.

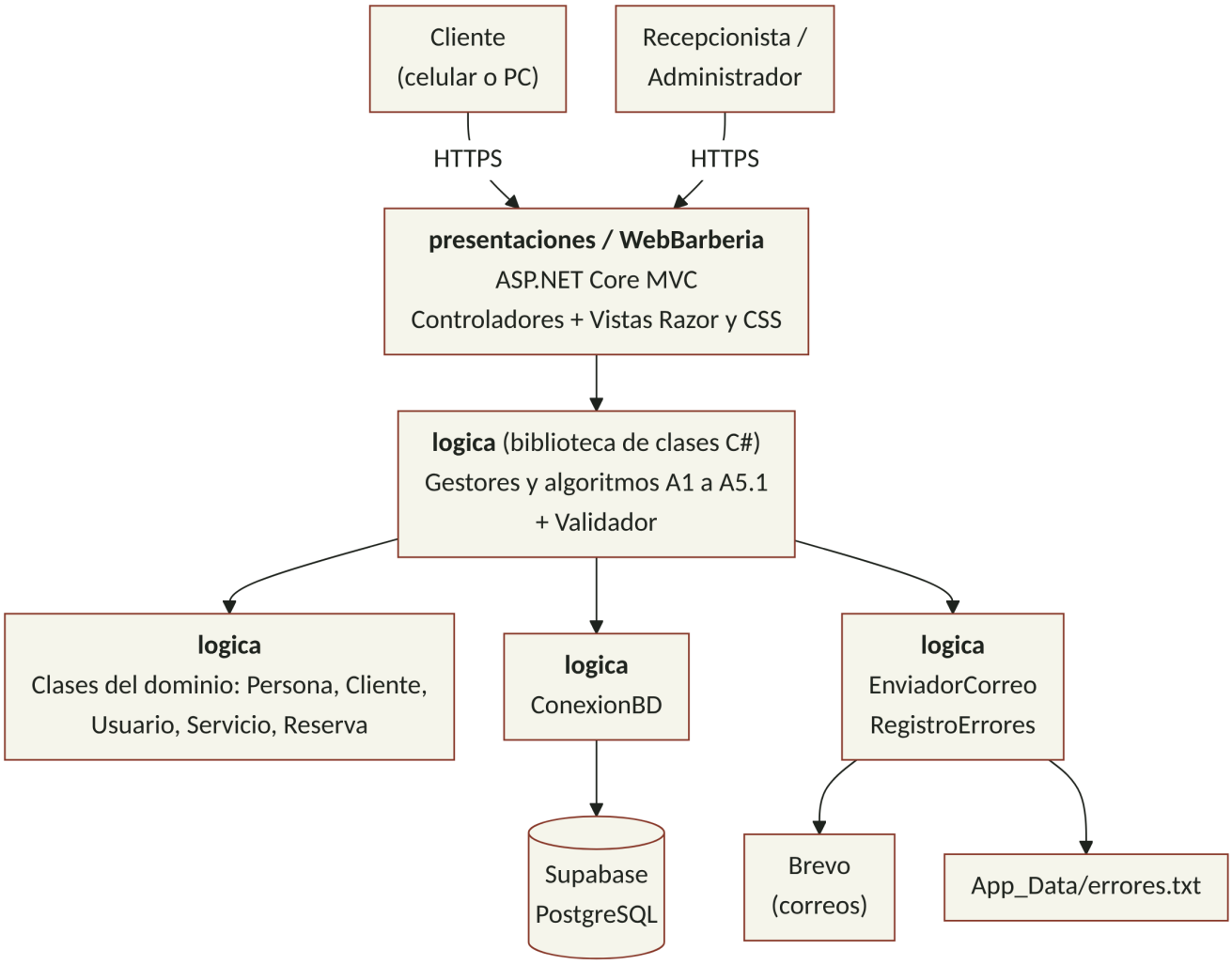


Figura 17. Arquitectura de la solución

B.2 Módulos y pantallas

Zona	Pantalla	Rol
Pública	Reservar cita (5 pasos: servicio, peluquero, fecha y hora, datos, confirmar) → ¡Cita confirmada!	Cliente
Pública	Acceso del personal (inicio de sesión)	—
Panel	Agenda del día y de la semana, con botón «Exportar .txt» (inicio del panel)	Recepcionista, Administrador
Panel	Reservas: nueva reserva manual, cancelar, atendida, no asistió, reprogramar	Recepcionista, Administrador

Zona	Pantalla	Rol
Panel	Clientes: lista, búsqueda, nuevo, editar, eliminar, historial	Recepcionista, Administrador
Panel	Servicios	Administrador
Panel	Peluqueros (con acceso a la agenda de cada uno)	Administrador
Panel	Usuarios	Administrador
Panel	Reportes y exportación .txt	Administrador
Panel	Registro de errores	Administrador
Panel	Cerrar sesión	Recepcionista, Administrador

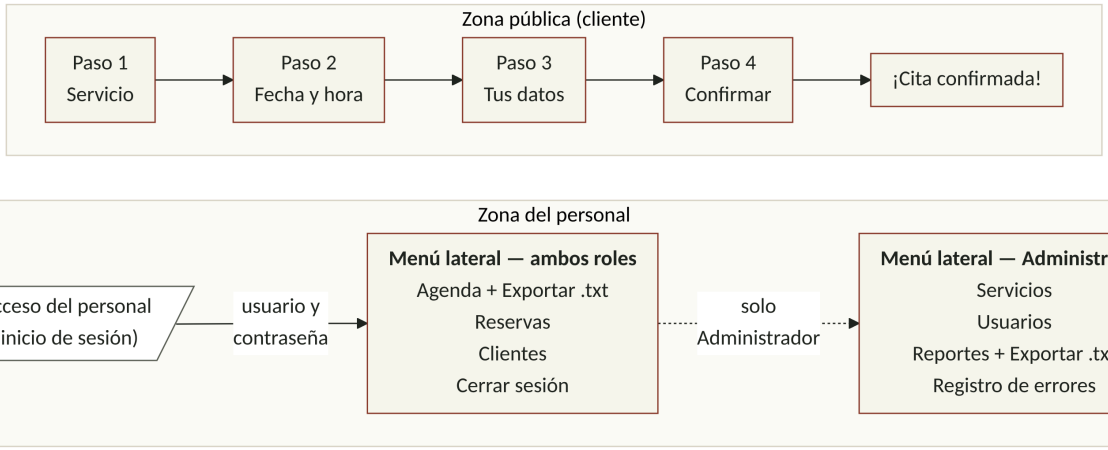


Figura 18. Mapa de navegación

B.3 Modelo de datos preliminar

Tabla	Campos principales
servicios	id, nombre (único), descripcion, duracion_min, precio, imagen, activo
peluqueros	id, nombre, apellido, especialidad, activo
clientes	id, nombre, apellido, telefono (único), correo, fecha_registro
reservas	id, cliente_id → clientes, servicio_id → servicios, peluquero_id → peluqueros, fecha, hora_inicio, hora_fin, precio, estado, notas, origen, correo_enviado, creado_en, creado_por → usuarios (opcional)
usuarios	id, nombre, apellido, nombre_usuario (único), contrasena_hash (texto "sal:hash"), rol, activo, intentos_fallidos, bloqueado_hasta

- **Relaciones:** un cliente tiene muchas reservas; un servicio aparece en muchas reservas; un peluquero atiende muchas reservas; un usuario puede registrar muchas reservas manuales.
- **Tipos:** fecha → date; hora_inicio y hora_fin → time (hora de Bolivia); creado_en, fecha_registro y bloqueado_hasta → fecha y hora sin zona horaria, con el valor puesto desde C# con `Reloj.AhoraBolivia()`; correo_enviado → lógico que admite vacío (vacío = la reserva no tiene correo).
- **Valores fijos (enumeraciones):** estado ∈ {Confirmada, Atendida, Cancelada, NoAsistio} (`EstadoReserva`; en pantalla, «No asistió»); rol ∈ {Administrador, Recepcionista} (`Rol`); origen ∈ {Web, Manual} (`OrigenReserva`).
- **Restricción anti-choques (garantiza RNF-07):** la base de datos rechaza guardar dos reservas Confirmadas o Atendidas **del mismo peluquero** que se superpongan (RN-06, RN-21), aunque lleguen al mismo tiempo. En PostgreSQL se escribe `EXCLUDE USING gist (peluquero_id WITH =, tsrange(fecha + hora_inicio, fecha + hora_fin, '[)') WITH &&) WHERE (estado IN ('Confirmada','Atendida'))`, que se lee: «no permitir dos filas con el mismo peluquero (`=`) cuyos rangos de fecha y hora [inicio, fin) se superpongan (`&&`), contando solo las

Confirmadas o Atendidas». Combinar `=` con `&&` necesita la extensión `btree_gist`. Si se intenta guardar un choque, PostgreSQL responde con el código 23P01 y el sistema lanza `HorarioNoDisponibleException`.

- **Datos iniciales:** script SQL con los 5 servicios referenciales y un peluquero; Administrador inicial creado por la aplicación si no hay usuarios.

B.4 Clases candidatas (el detalle va en la Presentación 2)

Clase	Responsabilidad
Persona (abstracta)	nombre y apellido; la heredan Cliente, Usuario y Peluquero
Cliente	teléfono, correo y fechaRegistro
Usuario	nombreUsuario, contrasenaHash (incluye la sal), rol, activo, intentosFallidos, bloqueadoHasta
Servicio	nombre, descripción, duración, precio, imagen, activo
Peluquero	especialidad, activo (agenda propia)
Reserva	cliente, servicio, peluquero, fecha, horaInicio, horaFin, precio, estado, notas, origen, correoEnviado, creadoEn, creadoPor (Usuario, opcional)
EstadoReserva, Rol, OrigenReserva	enumeraciones
GestorReservas	A1, A1.1, A2 y A4
GestorClientes, GestorServicios, GestorPeluqueros	CRUD de clientes (con búsqueda lineal), servicios y peluqueros
GestorUsuarios	A3 y CRUD de usuarios
GeneradorReportes	A5, A5.1 y exportación .txt
Validador	reglas de validación de los datos de entrada
Seguridad	calcular y verificar el hash PBKDF2
EnviadorCorreo	llamada a la API de Brevo
RegistroErrores	escribir y leer `errores.txt`
ConexionBD	abrir conexiones Npgsql con la cadena de configuración
Reloj	`AhoraBolivia()`
HorarioNoDisponibleException	excepción propia del choque de horario

Anexo C. Cumplimiento de requisitos y riesgos

C.1 Cumplimiento de los requisitos del módulo

Requisito del módulo	Dónde se cumple
Menú principal	Menú lateral del panel según el rol: Agenda, Reservas, Clientes, Cerrar sesión; y, solo para el Administrador, Servicios, Usuarios, Reportes y Registro de errores (S-05).
CRUD	Clientes (RF-16); Servicios (RF-18, con borrado lógico); Usuarios (RF-19); Reservas: crear (RF-06, RF-12), leer (RF-10), actualizar (RF-14, RF-15), anular (RF-13).
Archivos de texto	Guardar: `errores.txt` (RF-22), agenda (RF-21) y reportes (RF-20) en .txt. Cargar: lectura de `errores.txt` en el panel (RF-23).
Validaciones	Tabla de validaciones (sección 6) y reglas RN-02 a RN-09.
Excepciones	`try/catch` en A2, A4, A5 y en el registro de errores; excepción propia `HorarioNoDisponibleException`.

Requisito del módulo	Dónde se cumple
Complejidad algorítmica	A1: O(b × r); A1.1: O(r); A5: O(n + k^2) con burbuja; búsqueda de clientes: O(n) (recorrido lineal).
POO	Clases del Anexo B.4: encapsulamiento, constructores, herencia (Persona → Cliente, Usuario) y enumeraciones.
Estructuras de control	Secuenciales, condicionales (Si, Según) y repetitivas (Mientras, Para, Repetir … hasta que) en A1 a A5.1.

C.2 Riesgos técnicos y verificación

Código	Riesgo	Mitigación
R-01	El hosting gratuito no permite conectarse a Supabase, llamar a Brevo, escribir en App_Data o usar HTTPS.	**Prueba de humo antes del 13/10** (prueba mínima de que todo conecta): publicar una página que haga una consulta simple a Supabase, envíe un correo de prueba, escriba una línea en `App_Data/errores.txt` y cargue por HTTPS. Plan B: Render o Azure for Students.
R-02	Supabase gratis pausa el proyecto tras una semana sin uso.	Usar la base todos los días; revisar el panel y hacer una reserva de prueba el día antes de cada presentación (18/10, 22/10 y 25/10); guardar en el repositorio el script SQL de creación y exportar un respaldo.
R-03	El correo cae en Spam, es rechazado o Brevo bloquea la dirección del servidor.	Remitente verificado en Brevo; autorizar en Brevo la dirección IP del hosting; probar con Gmail y Outlook antes del 19/10; la reserva vale igual (RN-20) y la pantalla avisa si el correo no salió.
R-04	Lentitud: el hosting gratuito está en Europa.	Crear el proyecto de Supabase en una región europea (por ejemplo, Frankfurt), cerca del servidor; usar la conexión "Session pooler" con un máximo de 10 conexiones; abrir el sitio antes de presentar.
R-05	Reservas falsas desde el formulario público.	Riesgo aceptado: el personal las cancela desde el panel (RF-13).
R-06	Se pierde el registro de errores al volver a publicar.	En el perfil de publicación, excluir App_Data y no borrar archivos adicionales del servidor.

Anexo D. Glosario

Término	Significado
API	Forma en que un programa le pide algo a otro por internet; aquí, el sistema le pide a Brevo que envíe un correo.
ASP.NET Core MVC	Tecnología de Microsoft para hacer sitios web en C#, separando Modelos (datos), Vistas (páginas) y Controladores (lógica de cada pantalla).
Brevo	Servicio externo que envía correos electrónicos.
Complejidad O( )	Cómo crece la cantidad de pasos de un algoritmo según el tamaño de los datos; O(b × r) significa "aproximadamente b por r pasos".
Cookie	Pequeño dato que el navegador guarda para recordar que el usuario inició sesión.
CSRF	Ataque en el que otra página envía un formulario falso en nombre del usuario; se evita con el token antiforgery.
Enumeración (enum)	Tipo de dato con una lista fija de valores, por ejemplo los estados de una reserva.
Hash, sal y PBKDF2	El hash es una huella de la contraseña que no se puede revertir; la sal es un valor aleatorio que se mezcla para que dos contraseñas iguales den huellas distintas; PBKDF2 es el método que calcula el hash muchas veces para hacerlo lento de adivinar.

Término	Significado
HTTPS	Conexión cifrada entre el navegador y el servidor.
Inyección SQL	Ataque que mete órdenes de base de datos dentro de un dato; se evita con consultas con parámetros.
Npgsql	Biblioteca de C# para conectarse a PostgreSQL.
RLS	Seguridad por filas de Supabase: sin reglas, nadie puede leer las tablas desde internet.
Session pooler	Dirección de conexión de Supabase que funciona en servidores que solo usan IPv4.
Supabase / PostgreSQL	Servicio en la nube que aloja la base de datos PostgreSQL del sistema.
Transacción	Grupo de operaciones en la base de datos que se guardan todas juntas o ninguna.
XSS	Ataque que inyecta código en una página; se evita mostrando lo que escribe el usuario como texto.